Chair of Network Architectures and Services
School of Computation, Information, and Technology
Technical University of Munich



TECHNICAL UNIVERSITY OF MUNICH

SCHOOL OF COMPUTATION, INFORMATION, AND TECHNOLOGY

INFORMATICS

MASTER'S THESIS IN INFORMATICS

Building a Production-Ready Data Pipeline for Large-Scale Data Processing

M. Furkan Atasoy

Technical University of Munich

School of Computation, Information, and Technology

Informatics

Master's Thesis in Informatics

Building a Production-Ready Data Pipeline for Large-Scale Data Processing

Entwicklung einer produktionsbereiten Datenpipeline für die Verarbeitung von groß angelegten Datenmengen

Author:	M. Furkan Atasoy
Supervisor:	Prof. Dr.-Ing. Georg Carle
Advisor:	Filip Rezabek
	Kilian Glas
Date:	March 30, 2025

I confirm that this Master's Thesis is my own work and I have documented all sources and material used.

Garching, March 30, 2025

Location, Date



Signature

Abstract

Blockchain produces high volumes of complex and continuously evolving data, and it is a challenge to store and analyze it. There are existing solutions, such as public RPC clients and industry-standard analytics platforms like Dune Analytics. They provide valuable insights, but they lack access to real-time off-chain data as well as the flexibility researchers require. To address these limitations, this thesis presents a production-ready data pipeline designed for continuous blockchain data ingestion, efficient storage, and flexible user access.

The proposed system follows a customized ELT processing model for the ingestion of large-scale blockchain data. It utilizes a data lake structure and on-demand transformations to ensure scalability, maintainability, and adaptability to evolving blockchain data. By using MinIO as a scalable object storage system, DuckDB as a lightweight query engine, and JupyterHub as a user-friendly analytics platform, the pipeline enables researchers to access blockchain data in a flexible way. Additionally, it includes caching and monitoring layers to create a performance-efficient and observable platform.

To evaluate and show the effectiveness of the system, the thesis focuses on creating real-world blockchain workloads through workload emulations. The system is tested over long periods using these workload emulations that mimic user interactions. The results show that the pipeline achieves real-time ingestion latency, efficient resource utilization, and low-latency data querying through its efficient architecture. Additionally, the workload emulations are designed through a real-world use case: Maximal Extractable Value (MEV) analysis, which showcases the pipeline's capability to support complex blockchain research.

In conclusion, the proposed pipeline is capable of running over long periods while handling real-time blockchain data ingestion and serving multiple users simultaneously. Under simulated workloads involving 60 concurrent users and diverse query types, the system maintained ingestion latencies around 53 seconds and average query latencies as low as 16 seconds when caching was enabled. Thanks to its modular architecture, resource isolation, and caching, the pipeline scales effectively without overloading nodes, keeping total CPU usage under 80% even during peak traffic. This architecture ensures scalability, maintainability, and resource efficiency, making it suitable for a wide range of blockchain research applications. While the proposed pipeline meets the research objectives, there are still possible enhancements that can be done as future work, including expanding data sources, integrating advanced data management solutions, and optimizing caching strategies to further enhance usability and performance.

Contents

1 Introduction **1**
- 1.1 Goals . . . 1
- 1.2 Research Questions . . . 2
- 1.3 Outline . . . 2

2 Background **5**
- 2.1 Blockchain Overview . . . 5
  - 2.1.1 Fundamental Principles . . . 5
  - 2.1.2 On-Chain Data . . . 6
  - 2.1.3 Off-Chain Data . . . 7
- 2.2 Data Storage and Processing Solutions . . . 8
  - 2.2.1 Data Architectures . . . 8
  - 2.2.2 Data Storage Architectures . . . 10
  - 2.2.3 Data Transformation Techniques . . . 11
- 2.3 Data Access Solutions . . . 12
  - 2.3.1 Communication Protocols . . . 12
  - 2.3.2 Authentication and Authorization Protocols . . . 13
  - 2.3.3 Centralized Logging and Monitoring . . . 14
- 2.4 System Evaluation Overview . . . 15
  - 2.4.1 Evaluation Metrics . . . 15
  - 2.4.2 Queuing Theory . . . 15
  - 2.4.3 Open and Closed Benchmarking . . . 16

3 Related Work **17**
- 3.1 Blockchain Data Pipelines in Academia . . . 17
  - 3.1.1 Data Collection Pipeline for Blockchain Platforms . . . 17
  - 3.1.2 TUMChainBook . . . 18
- 3.2 Industry-Standard Blockchain Data Processing Solutions . . . 20

3.2.1 RPC Clients . . . 20
3.2.2 Dune Analytics . . . 20
3.2.3 The Graph Network . . . 21

4 Analysis **23**
4.1 Introduction to Use Case . . . 23
4.1.1 Overview of MEV Analysis . . . 23
4.1.2 Importance and Relevance of MEV Analysis . . . 24
4.1.3 Key Analytics for MEV Analysis . . . 24
4.2 Technical Causes . . . 25
4.2.1 Complexity of Blockchain Data . . . 25
4.2.2 Limited Existing Solutions . . . 25
4.3 Requirements Analysis . . . 27
4.3.1 Functional Requirements . . . 27
4.3.2 Non-Functional Requirements . . . 28
4.4 Technical Analysis . . . 28
4.4.1 Blockchain-Specific Analysis . . . 29
4.4.2 Data Storage and Processing Analysis . . . 29

5 Design **35**
5.1 Architectural Overview . . . 35
5.2 Layers . . . 36
5.2.1 Extract Layer . . . 37
5.2.2 Load Layer . . . 38
5.2.3 Transformation and Access Layer . . . 39
5.2.4 Monitoring Layer . . . 41

6 Implementation **45**
6.1 Code Structure . . . 45
6.1.1 Infrastructure . . . 45
6.1.2 Modules . . . 45
6.1.3 Services . . . 45
6.2 Infrastructure . . . 46
6.2.1 Node Deployments . . . 46
6.2.2 Service Deployments . . . 49
6.2.3 Service Configurations and Resource Limitations . . . 51
6.2.4 External Access . . . 52
6.3 Iterative Development . . . 52
6.3.1 Version 1 . . . 53

6.3.2 Version 2 . . . 54
6.3.3 Version 3 . . . 54
6.3.4 Version 4 . . . 55

7 Evaluation **57**
7.1 Evaluation Metrics . . . 57
7.1.1 Ingestion Metrics . . . 58
7.1.2 Query Metrics . . . 58
7.1.3 Resource Usage Metrics . . . 58
7.2 Workload Emulation Setup . . . 58
7.2.1 User Traffic . . . 58
7.2.2 User Personas . . . 60
7.2.3 Workload Diversity . . . 60
7.2.4 Cache Behavior . . . 60
7.3 Workload Emulation Results . . . 61
7.3.1 Data Growth . . . 62
7.3.2 Ingestion Latency and Throughput . . . 63
7.3.3 CPU and Memory Usage . . . 64
7.3.4 Identified Bottlenecks . . . 65
7.4 Performance Improvements . . . 66
7.4.1 Reducing Load on Bottlenecks . . . 66
7.4.2 Preventing Resource Overload . . . 66
7.5 System Comparison . . . 67
7.5.1 Impact of NFS Storage . . . 67
7.5.2 Impact of Caching . . . 68
7.6 Discussions . . . 70
7.6.1 Scalability and Requirements Fulfillment . . . 70
7.6.2 Long-Term Operation and Planning . . . 71

8 Conclusion **73**
8.1 Research Questions . . . 73
8.2 Future Work . . . 75
8.2.1 Expanding Data Sources . . . 75
8.2.2 Metadata Management and Improved Data Exploration . . . 75
8.2.3 Dynamic Caching Strategies . . . 76
8.3 List of acronyms . . . 77

Bibliography **79**

List of Figures

3.1 Architecture of Ayvaci's Blockchain Data Collection Pipeline 18
3.2 Architecture of TUMChainBook 19

5.1 Simplified Architecture 36
5.2 Architecture: Extract and Load Layers 37
5.3 Architecture: Transformation and Access Layers 39
5.4 Architecture: Monitoring Layer 42

6.1 Pipeline Folder Structure 46
6.2 Physical Node Deployments 47
6.3 Iterative Development of the System 53

7.1 Data Growth Over Time 62
7.2 Ingestion Latencies by Data Source Over Time 63
7.3 Ingestion Throughput by Data Source Over Time 64
7.4 Query Latency Over Time 65
7.5 CPU Usages by Services Over Time 65
7.6 Query Latency Over Time (with Caching) 68
7.7 CPU Usages by Services Over Time (with Caching) 69
7.8 Cache Hit Rate Over Time 69

List of Tables

7.1 Ingestion Latency and Throughput Metrics 62
7.2 CPU and Memory Usage Metrics 64
7.3 Impact of NFS and Caching on Ingestion and Query Latencies 67
7.4 Impact of Caching on Resource Consumption 68

Chapter 1

Introduction

Blockchain technology has transformed digital transactions, governance, decentralized finance, and many other fields. As a result, researchers want to analyze it and have a better understanding of its nature to be able to utilize it correctly. However, blockchain produces a high volume of complex data, and it constantly evolving. Therefore, it is a challenge to develop a system that can continuously ingest blockchain data and serve it to researchers with a user-friendly platform.

There are existing solutions aiming to create such an analytical platform. However, they lack in certain aspects. Public RPC clients provide direct access to blockchain state and real-time data, but they are often restricted to on-chain information and don't provide off-chain data querying out of the box. On the other hand, there are several industry-standard solutions that enable efficient querying of on-chain and off-chain data, such as Dune Analytics [1]. However, they lack flexibility for custom processing steps and are mainly community-driven. Therefore, researchers require a platform on which they can run their analyses in a flexible way.

1.1 Goals

The primary goal of this thesis is to design and implement a production-ready data pipeline that can continuously ingest both on-chain and off-chain blockchain data, then integrate it with a JupyterHub platform to provide a user-friendly platform for researchers to run their analyses flexibly. In addition, to show the effectiveness of the pipeline and have a user-centric approach, the thesis aims to implement real-world research use cases throughout the pipeline's development.

1.2 Research Questions

The core research questions guiding this thesis focus on the design, implementation, and evaluation of a scalable blockchain data pipeline.

RQ1: *How can we design and deploy a pipeline that can continuously ingest blockchain data over long periods?*

We aim to develop a production-ready data pipeline, and in order to achieve production-readiness, we need to be sure that it can run over long periods without interruptions while ensuring reliability, scalability, and efficient storage management.

RQ2: *Which established technologies and solutions should be used to make the pipeline robust and production-ready?*

A production-ready data pipeline should have certain promises such as maintainability, observability, scalability, etc., and it is hard to ensure them by not using industry-standard, established solutions. Therefore, the pipeline should use necessary established technologies to ensure ease of maintenance while being scalable and reliable across varying loads.

RQ3: *What is the most suitable architecture for building a large-scale data collection pipeline?*

It is important to use established solutions, but it is a challenge to integrate them in a way that they can perform efficiently. Therefore, the thesis should focus on finding out the most suitable architecture for orchestrating established solutions.

RQ4: *Which representative use cases are suitable for demonstrating the effectiveness of the pipeline?*

While developing a complex system, it is a common fault to miss what users really want and solve non-existent problems. To avoid that, the thesis should have a use-case centric approach and develop a pipeline with real-world use cases.

1.3 Outline

This thesis is structured as follows: Chapter 2 provides background on blockchain data processing, data engineering principles, and relevant software components. Chapter 3 discusses the existing solutions. Chapter 4 outlines the challenges of handling blockchain data and defines the requirements for the proposed pipeline. Chapter 5 presents the architectural design, detailing its modular structure and key components. Chapter 6 describes the implementation, including infrastructure setup and optimizations. Chapter

7 evaluates the pipeline's performance through workload emulations, analyzing ingestion latency, query efficiency, and resource utilization. Finally, Chapter 8 summarizes findings, addresses research questions, and suggests future improvements.

Chapter 2

Background

The efficient management and analysis of blockchain data require an understanding of both its fundamental principles and the underlying data processing techniques. This chapter provides an overview of blockchain technology, its data structure, and the distinction between on-chain and off-chain data. Additionally, it explores modern data storage and processing solutions, system communication solutions, and system evaluation that are essential for building a scalable and efficient blockchain data pipeline.

2.1 Blockchain Overview

This section introduces the core principles of blockchain, discusses the structure and characteristics of on-chain and off-chain data, and highlights the challenges associated with data storage and processing in decentralized systems.

2.1.1 Fundamental Principles

Blockchain is a decentralized digital ledger that creates immutable and transparent records and distributes these records to multiple nodes to allow decentralization. This enables nodes to communicate directly without relying on a central authority [2], [3].

Decentralization: Unlike traditional systems where a central authority, such as banks and governments, is required to provide trust, in Blockchain, we rely on a peer-to-peer network to verify the information [3]. This process is done with the help of the consensus mechanism of blockchains, which enables participants to agree on the validity of the transactions without relying on a third-party actor [2].

Immutability: Transactions recorded in the blockchain cannot be altered. This is ensured by linking each block to the previous one using a cryptographic hash. For instance, in the Proof of Work (PoW) mechanism, this linkage makes it computationally infeasible for an attacker to recompute all subsequent blocks.

Transparency: One of the key promises of blockchain systems is full transaction visibility, allowing anyone to view the transaction history. While this ensures transparency, it also maintains pseudonymity, as users are identified by public keys rather than personal identities [4].

2.1.2 On-Chain Data

On-chain data refers to all information that is permanently recorded on the blockchain ledger and validated through its consensus mechanism [5]. Due to the immutable and decentralized nature of blockchain, once data is added, it cannot be altered or deleted, ensuring transparency and security.

Examples of On-Chain Data

Transaction: A transaction refers to any operation recorded on the blockchain ledger. For example, a Bitcoin transaction transfers BTC between two addresses, while an Ethereum transaction can involve interactions with smart contracts, such as calling their functions [6].

Event Log: In Ethereum, interacting with a smart contract can generate various types of event logs. These logs are recorded on the blockchain, making them on-chain data. For example, when an ERC-20 token [7] is transferred from one account to another, the smart contract creates a transfer event, which is then permanently stored in the blockchain.

Block: Block is another fundamental data structure of blockchain networks. It is created by the validation and consensus mechanisms and then linked to the previous block through the cryptographic hash. In Ethereum, a block contains fields such as transactions, metadata, cryptographic hashes, and event logs.

Challenges of On-Chain Data

Due to its immutable, decentralized, and append-only nature, on-chain data presents unique challenges. One of the examples of these challenges is storage constraints.

Blockchain networks grow continuously due to two key factors. Firstly, transaction volumes increase daily, resulting in growing demands for storage. Secondly, every trans-

action executed remains permanently recorded in the chain, unlike traditional databases that allow the deletion of old data [8].

As blockchain data continues to grow, it must be replicated and stored on each participating node, further increasing storage demands. Bitcoin's blockchain has surpassed 350 GB, while it is even larger for Ethereum due to its smart contract activities, resulting in more than 1 TB of data as of now. This challenges the node operators because they must allocate an increasing amount of disk space and computation power [8].

As blockchain scales, solutions like pruning, off-chain data storage, and Layer-2 solutions are developed to manage storage limitations [9].

2.1.3 Off-Chain Data

Off-chain data includes any information related to blockchain operations that is not stored directly on the blockchain ledger. It is essential for the network's functionality, providing real-time updates and supporting overall system performance. Unlike on-chain data, which is permanently recorded, off-chain data consists of temporary or infrastructure-related information [5].

Off-chain data can be more resource-intensive to process than on-chain data because it updates frequently and is not permanently recorded on the blockchain ledger. To ensure a comprehensive understanding of blockchain activity, it is crucial that third parties continuously collect and store this data [8] and serve it to researchers so they can conduct their analyses.

Examples of Off-Chain Data

Mempool Data (Pending Transactions): The mempool (memory pool) is a temporary place where transactions are waiting for a confirmation to be included in a block. A transaction is in a mempool when it is being broadcast to the network but still not validated and included by a validator/miner. In Ethereum, a transaction in the mempool includes fields like transaction hash, sender and receiver addresses, and gas prices.

P2P Network Data (Node Communication & Propagation): Blockchain networks rely on a peer-to-peer architecture, where nodes communicate directly to exchange messages. This P2P communication generates off-chain network data, which includes latency and propagation speed —measuring how quickly transactions and blocks are broadcasted— peer discovery data, which tracks new nodes joining and leaving the network, and network topology, which provides insights into node connectivity and the overall decentralization of the network.

Price Data from Centralized & Decentralized Exchanges: Cryptocurrency price data is an important type of off-chain data. It is managed and stored by centralized and decentralized exchanges, making it accessible to blockchain applications through oracles or external APIs [5]. This data includes real-time asset prices, trading volumes, historical market trends, and other market-related insights, which are essential for financial analysis and smart contract interactions.

Challenges of Off-Chain Data

Since off-chain data is not permanently stored on the blockchain, it doesn't inherit the promises of blockchain, such as data availability, security, and trust, so it is subject to loss, inconsistency, and also manipulation of data. Therefore, it is critical to understand its potential security risks as well as its availability issues. Collecting and storing off-chain data independently can provide greater control over its accessibility and integrity, eliminating the need to rely on third-party providers [10].

2.2 Data Storage and Processing Solutions

The efficient storage and processing of blockchain data are crucial for ensuring scalability, accessibility, and performance in large-scale analytical systems. This section explores different data architectures, storage solutions, and transformation techniques that enable high-performance data processing.

2.2.1 Data Architectures

This section explores common data architectures, including data lakes, data warehouses, and hybrid models, highlighting their strengths, use cases, and trade-offs in handling structured and unstructured blockchain data.

Data Lakes

Data lakes store data that can be structured, semi-structured, or unstructured. It doesn't give any promises about the schemas of data, which makes it flexible in data ingestion. Because there is no structure needed, it also minimizes the data loss, which can happen while transforming data. Data lakes usually use raw data formats such as JSON, Parquet, and CSV, depending on the pipeline's requirements [11].

Schema-on-read is the fundamental property of the data lake, which aims to store data in its native format and structure when it is queried. This provides greater flexibility, but it has a disadvantage in that the query latency can be slower compared to other storage solutions.

The biggest advantage of the data lakes comes from its core design principle, which separates storage from computation, allowing them to focus only on data storage [12]. This makes data lakes well-suited for cloud-native environments, as they enable horizontal scaling by allowing storage and processing power to be scaled independently. They are often used alongside distributed computation frameworks such as Apache Spark and Hadoop to efficiently process large-scale datasets [13], [14].

Data lakes are primarily used by Data Scientists and Machine Learning (ML) engineers who require raw data access for model training without the constraints of predefined schemas.

Data lakes are primarily used inside of an Extract, Load, Transform (ELT) processing model, which begins by extracting raw data from various sources, then continues by loading it into a data lake, and finally transforming it as needed by users.

Several established solutions are widely used in the industry for data lakes. One example is Hadoop Distributed File System (HDFS), an open-source project designed to work with Apache Hadoop for distributed data processing [14]. Over time, Hadoop's processing framework has largely been replaced by Apache Spark [13]. However, with the rise of cloud-based systems, HDFS is now considered a legacy solution, and companies increasingly prefer cloud-native object storage systems like S3 [15] and MinIO [16]. These modern storage solutions are easier to run with cloud-native orchestration frameworks such as Kubernetes, providing better scalability and flexibility.

Data Warehouses

Data warehouses store structured and processed data that has already been formatted for analytical workloads. They utilize indexing and partitioning to provide high-speed access to structured historical data, which makes them ideal for business intelligence and reporting [17].

They use the schema-on-write paradigm, which requires data to be structured, validated, and processed before it is stored. In this way, it ensures consistency and is optimized for querying. On the other hand, this paradigm adds additional preprocessing overhead, limits the flexibility, and results in a loss of information while processing and transforming the raw data [18].

Business Intelligence (BI) teams are the primary users of data warehouses since their primary responsibility is to analyze data to help the company in decision-making processes that extensively need structured, historical data.

Their primary processing model is Extract, Transform, Load (ETL), ensuring data is extracted and preprocessed, validated, cleaned, and optimized for efficient historical access before being loaded into a data warehouse.

There are many established solutions for data warehouses such as managed solutions like Amazon Redshift [19] and Snowflake [20], [21] as well as open-source solutions like Clickhouse [22].

Hybrid Models

Hybrid models combine data lakes and data warehouses to leverage the flexibility of raw data storage while also ensuring structured, optimized querying for analytics [23].

They support both schema-on-write and schema-on-read, allowing saving raw data to a data lake and then processing and transforming this data and finally storing it in a data warehouse. This hybrid model offers scalability and efficiency at the same time by allowing storing raw data in a cost-effective data lake while transforming frequently accessed, mission-critical data to structured data and allowing efficient query access.

Hybrid solutions are widely used in cloud environments, allowing organizations to provide raw, unstructured data for data scientists while offering structured historical data for business intelligence teams. This approach enables more efficient and organized data management.

However, integrating two complex systems presents challenges, as managing data consistency and governance becomes more difficult. To address these issues, organizations often implement additional solutions for data lineage and metadata management, which can further increase system complexity and maintenance effort. Therefore, it is essential to carefully assess system requirements before adopting a hybrid solution.

One example of hybrid models is Databricks' Lakehouse Platform [24], which combines a data lake and a data warehouse into a unified Lakehouse architecture to support structured analytics and AI/ML workloads at the same time.

2.2.2 Data Storage Architectures

Data storage architecture defines how data is stored, accessed, and managed within a system. Depending on the requirements and the workload of a system, there are different storage solutions. In this section, examples of different open-source storage architectures are discussed.

MinIO

MinIO is a high-performance, S3-compatible object storage system designed for large-scale data storage and it is commonly used in cloud-native applications [16].

MinIO is designed for high-speed data processing and flexible raw data storage. It has built-in redundancy and high availability thanks to its distributed architecture. It has out-of-the-box support for Apache Spark, and DuckDB with its S3-compatible API, and it easily integrates with Kubernetes thanks to its distributed architecture.

HDFS (Hadoop Distributed File System)

HDFS is a distributed file system designed to store and process large datasets across multiple machines. It has a shared file access feature that enables multiple clients to read and write data concurrently from a remote file system [14].

However, HDFS is built for clusters where storage and computing are closely linked. It is mostly used in bare-metal or virtual machines. Because of that, it doesn't perform well with a standard cloud environment and requires special configurations to be able to work efficiently, which makes it not an ideal solution for a cloud-native environment and orchestration solutions like Kubernetes.

Network File System (NFS)

NFS is a file-based storage system that allows multiple clients to access files stored on a central storage pool over a network. It employs a client-server model, where an NFS server handles the incoming requests and serves respective files and NFS clients access shared files by mounting the respective folders to their local machine, and in their point of view, these mounts are not different than local files, which makes accessing to files straightforward [25].

2.2.3 Data Transformation Techniques

Data transformation is a crucial step in data pipelines, allowing raw data to be cleaned, structured, and optimized according to the data access workload. There are different available solutions based on their scale, performance, and integration capabilities. Two widely used solutions are discussed in the following sections because of their relevance to our topic.

DuckDB

DuckDB is an embedded, lightweight SQL engine designed for analytical processing, particularly for processing raw data such as CSV and Parquet. Unlike traditional databases, DuckDB doesn't employ a storage service and it only focuses on the processing

part, which makes it a good fit for architectures that want to decouple storage and computation. It runs entirely in-process, making it lightweight and easy to manage [26].

It integrates seamlessly with S3-compliant object storage to get raw data and store it efficiently by using SQL statements, which makes it easy to use since most developers are familiar with SQL statements, so it doesn't have a steep learning curve.

However, it comes with its limitations. Because it is an in-process, lightweight solution, it is not suitable for highly distributed, petabyte-scale workloads that require multi-node execution.

Apache Spark

Apache Spark is a distributed data processing engine that works on large-scale transformation and analysis across multiple computing nodes. It is designed for big data workloads and, similarly to DuckDB, it does not employ a storage service, instead, it integrates with data lake solutions for data access [13].

Sparks uses a similar paradigm with systems like Hadoop MapReduce, which breaks large datasets into smaller partitions and processes them in parallel. However, it outperforms Hadoop since it doesn't use disk-based storage instead, it uses an in-memory computing model, which makes the overall processing a lot faster.

It provides more detailed APIs, including Spark SQL, DataFrame API, and Spark Streaming, resulting in easy usage by developers.

Because of its complex modal and high-performance needs, it is a complex, resource-heavy system that consumes high memory and processing power and it is also not straightforward to configure it to a specific workload because of its underlying complexity.

2.3 Data Access Solutions

For a blockchain data pipeline to be effective, it needs reliable communication protocols and secure access control mechanisms. This section covers essential technologies that enable seamless data exchange, user access management, and system monitoring.

2.3.1 Communication Protocols

Representational State Transfer (REST)

REST is a web communication protocol that enables stateless, client-server interactions and it is commonly used in modern web applications [27].

It has a stateless communication mechanism, meaning each client request contains all the necessary information, eliminating the need for the server to store session data. This reduces server complexity and simplifies horizontal scaling, as additional server instances can be added to handle increased load.

WebSocket

WebSockets provide a full-duplex communication channel over a single TCP connection, which results in a real-time, bidirectional data transfer between the client and server [28].

It uses persistent connection, unlike REST, which increases the complexity of the server and limits the scaling but it reduces overall latency for real-time interactions. Therefore, it is commonly used for real-time applications.

2.3.2 Authentication and Authorization Protocols

OAuth 2.0

OAuth 2.0 is an authorization framework that allows third-party clients to access a user's resources without sharing credentials [29]. It is usually used with JSON Web Token (JWT), and thanks to JWT's stateless nature, it allows resource owners to authenticate users without requiring a persistent session.

It has advantages like scalability, interoperability, and reduced risks because the responsibilities of storing users' credentials and authenticating/authorizing them are taken from third-party users and given to safe systems such as Gmail and GitLab. However, it also has some challenges because it considerably increases the complexity of the app-side implementation, requiring multiple steps for authentication and authorization.

Role-Based Access Control (RBAC)

RBAC is an authorization model where permissions are assigned to specific roles rather than individuals, simplifying user access management in large-scale systems [30]. Roles are defined based on job functions, such as admin, editor, and viewer, each with specific permissions like read, write, and delete. Users are then assigned roles, granting them the corresponding permissions.

It has benefits like simplifying access control management by decoupling users from permissions and improving security with a proper principle pattern, such as the principle of least privilege, which ensures users only have the necessary access to perform their actions. In this way, it improves the overall security of the system.

2.3.3 Centralized Logging and Monitoring

Metrics

Metrics provide quantitative measurements that help track the performance and health of the applications. These include general resource utilization, such as memory and CPU usage, as well as latency across different parts of the system.

Prometheus

Prometheus is an open-source metrics collection system that uses time-series databases to store the metrics and a specialized query language (PromQL) for querying and aggregating the stored metrics data [31]. It also has built-in Kubernetes support, which makes it easier to use in cloud-native environments.

Logs

Logs provide detailed, unstructured data about system events, errors, and operations. They are useful for debugging and audit trails. There are two popular open-source stacks that are used by many software.

Loki + Fluentd Stack:

Loki is a horizontally scalable log aggregation system that stores logs and allows them to be searched with its indexing mechanism [32]. It is specifically designed to be lightweight, efficient and easy to configure since the monitoring part of a system is not the most critical part of it and it shouldn't spend a considerable amount of resources. However, its indexing is limited to metadata only so it is not able to perform full-text searches.

Fluentd [33] is an open-source data collector that is used to get data from various sources, such as Kubernetes pods, and deliver it to a log aggregation system such as Loki.

Loki and Fluentd together make a good combination, of lightweight log collection and aggregation systems.

Elasticsearch + Logstash + Kibana (ELK) Stack:

In the ELK stack, Elasticsearch performs storing and indexing of logs and it is a powerful engine capable of full-text search and has a horizontally scalable architecture [34]. However, it is a resource-intensive system and hard to configure for specific workloads. Logstash [35], similar to Fluentd, does the log collection part from the different sources and Kibana is a data visualization and exploration tool designed to work natively with Elasticsearch.

Together, they make a good combination for a powerful and full-text capable log collection and visualization system [36].

Visualization

Grafana is an open-source analytics and visualization platform used for the visualization of metrics and logs [37]. It can natively be integrated with various sources such as Prometheus for metrics and Loki for logs, providing single monitoring dashboards for metrics and logs. Therefore, it can complete the Loki + Fluentd stack as their visualization part and even expand that to include the metrics from Prometheus.

2.4 System Evaluation Overview

Evaluating the performance of a blockchain data pipeline is essential to ensure its efficiency, scalability, and reliability. This section covers evaluation metrics that help assess system performance, queuing theory that models how requests are processed over time, and benchmarking methods that simulate real-world workloads to identify potential bottlenecks.

2.4.1 Evaluation Metrics

Evaluation metrics are crucial in workload emulations as they provide quantifiable measurements to assess system performance and resource efficiency [38]. They have the following purposes:

1. Identifying how efficiently system resources are being used.
2. Finding the bottlenecks of the system.
3. Planning the capacity of the infrastructure.
4. Comparatively analyzing different possible architectures.

There are many possible evaluation metrics such as latency, throughput, and resource utilization.

2.4.2 Queuing Theory

Queuing theory is a mathematical framework used to model how requests arrive, wait, and are processed by systems. It is essential to simulate realistic workloads in order to perform performance analysis. Exponential interarrival times are commonly used to represent random, independent task arrivals which is typical in web traffic. They assume a memoryless process, where the likelihood of a request arriving is unaffected by previous arrivals [39].

2.4.3 Open and Closed Benchmarking

Open benchmarking simulates workloads where requests arrive continuously, regardless of the system's processing state. It is ideal for stress testing systems like web servers or query processing tools by focusing on their scalability limits [40].

On the other hand, closed benchmarking limits new requests until prior ones are processed, ensuring constant concurrency. It is commonly used in batch processing tools and provides predictable performance insights, but may underestimate bottlenecks if a system has a highly dynamic environment [40].

Chapter 3

Related Work

Understanding existing blockchain data processing solutions is essential for developing an efficient and scalable pipeline. This chapter explores previous research and industry-standard approaches to blockchain data collection, storage, and analysis. By exploring these works, we identify their strengths and limitations, helping shape the design of our proposed system.

3.1 Blockchain Data Pipelines in Academia

Academic research has contributed to the development of blockchain data pipelines by designing scalable architectures for data collection and processing. These solutions focus on efficient ingestion, storage, and accessibility of blockchain data for analysis.

3.1.1 Data Collection Pipeline for Blockchain Platforms

Data Collection Pipeline for Blockchain Platforms [41] is a master's thesis authored by Ilteber Ayvaci. The thesis focuses on designing a scalable, modular, and extensible pipeline capable of ingesting and processing blockchain data efficiently across multiple networks, including Ethereum and Algorand. The system was designed to efficiently manage data ingestion, storage, and querying, with a focus on optimizing throughput and reducing latency.

The architecture of the proposed pipeline follows a multi-layered approach, as illustrated in Figure 3.1, ensuring that blockchain data flows efficiently from data sources to end users. The core components of the pipeline include:

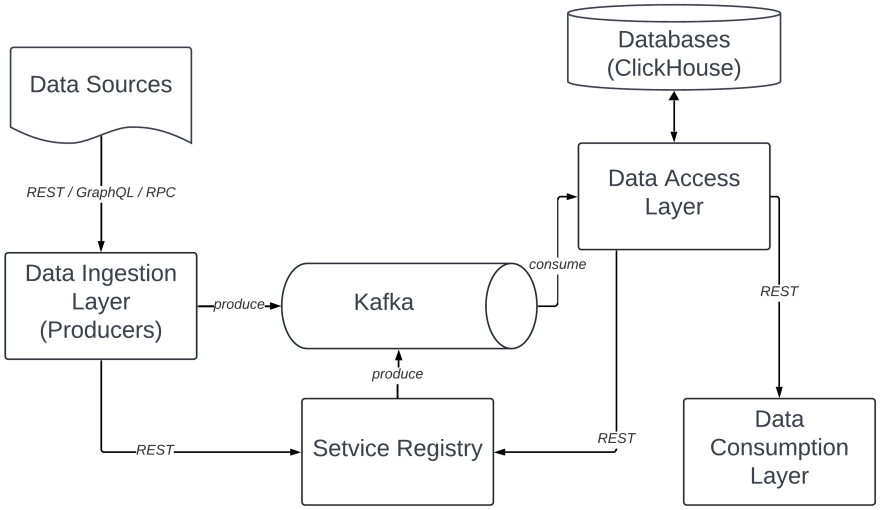


Figure 3.1: Architecture of Ayvaci's Blockchain Data Collection Pipeline

Data Ingestion Layer: It collects blockchain data through REST, GraphQL, and RPC APIs and processes it using Kafka-based message queues for efficient buffering and scalability.

Kafka: It serves as the backbone for decoupling producers from consumers, ensuring that data ingestion and storage operate asynchronously without performance bottlenecks.

Data Access Layer: It manages interactions with the ClickHouse database, optimizing queries for analytical workloads.

Service Registry: It maintains metadata about services interacting with the pipeline, enabling seamless integration of new data sources.

Data Consumption Layer: It provides REST API access for querying processed blockchain data.

This thesis played a foundational role in shaping our own work. In Section 4.2.2, its weak points are discussed in detail.

3.1.2 TUMChainBook

TUMChainBook is a multi-user Jupyter Notebook platform, designed specifically for working with blockchain data. It has multiple components integrated to create an

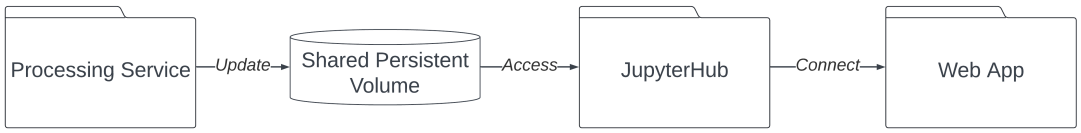


Figure 3.2: Architecture of TUMChainBook

environment in which researchers can perform their analysis. Figure 3.2 illustrates its components and their interactions.

As its core, TUMChainBook leverages JupyterHub, which consists of three components. Jupyter Notebook is its front-end interface where users interact with and execute their analysis. Jupyter Server is its back-end computation engine that processes user-defined code in an isolated environment. JupyterHub is an orchestration layer that manages multiple Jupyter Server instances, ensuring each user operates within their own dedicated runtime environment. For scalability and production deployment, TUMChainBook uses Zero to JupyterHub with Kubernetes, a production-ready distribution of JupyterHub [42]. This distribution provides a scalable microservice architecture where each component can be scaled independently.

TUMChainBook includes a Processing Service responsible for fetching blockchain data. It is a quite simple service that is implemented for demo purposes. It crawls Ethereum blockchain data daily and stores it in CSV format in the Shared Persistent Volume component.

Shared Persistent Volume is a central repository for blockchain data. It is basically a persistent volume in Kubernetes that is mounted to all Jupyter Servers in read-only fashion. In this way, it allows users to access blockchain data through their user interface.

TUMChainBook implements a custom web application that is specialized for blockchain data analysis. It consists of two components. The frontend is designed for blockchain data exploration by integrating interactive tools. It encapsulates the Jupyter Notebook implementation. On the other hand, the backend manages authentication via OAuth integration with GitLab, ensuring secure access to the platform.

TUMChainBook serves as an early-stage blockchain analytics platform, which is used as the foundation work of this thesis. In our work, it is used as a frontend component of the data pipeline. In Chapter 5, it is detailedly explained.

3.2 Industry-Standard Blockchain Data Processing Solutions

Blockchain data processing has evolved with several industry-standard solutions that enable efficient querying and analysis of blockchain data. These solutions range from RPC clients, which provide direct access to blockchain state and real-time data, to fully managed analytics platforms like Dune Analytics and The Graph Network. This section explores these approaches.

3.2.1 RPC Clients

RPC (Remote Procedure Call) clients are crucial components in blockchain data processing, providing a standardized way to interact with blockchain networks. These clients act as intermediaries between blockchain nodes and external applications by allowing an efficient retrieval of blockchain data. RPC clients provide access to both historical on-chain data and real-time off-chain data. Users can query these clients to retrieve information about the blockchain state, including specific transactions, blocks, and smart contract states. However, while on-chain data is available for historical queries, off-chain data can only be accessed in real-time. To support this, RPC clients use WebSocket connections to stream real-time off-chain data, such as mempool transactions, and use their P2P layer to retrieve peer-to-peer network data.

In the context of this thesis, RPC clients serve as the primary data source for the ingestion layer of our pipeline. Our data collection approach relies on Erigon, an Ethereum full node implementation, to fetch real-time on-chain and off-chain data.

3.2.2 Dune Analytics

Dune Analytics is a widely used blockchain analytics platform that enables users to query and visualize blockchain data through an SQL-based interface. It provides a user-friendly solution for on-chain data analysis, allowing researchers, traders, and developers to extract information without managing complex infrastructure.

Dune aggregates blockchain data from multiple networks and stores it in a structured relational database. Instead of making direct queries to blockchain nodes, users interact with pre-indexed datasets, which significantly reduces query latency. Users can write SQL queries to fetch specific blockchain data, and they visualize their results in interactive dashboards. It is an inspirational work for our thesis, and we use its core logic to fetch and serve blockchain data but instead of limiting users to SQL queries, we aim to provide Jupyter Notebooks, which can run more diverse analytics and use cases. The limitations are discussed in detail in Section 4.2.2.

3.2.3 The Graph Network

The Graph Network is a decentralized indexing and querying protocol designed for efficiently retrieving blockchain data, particularly from Ethereum and other EVM-compatible chains. It allows developers to create subgraphs, which define how data is indexed and structured for fast and efficient querying. By providing an API-like experience for blockchain data, The Graph Network simplifies decentralized application development and analytics.

The Graph allows developers to create custom subgraphs, which define specific blockchain data points. These subgraphs preprocess blockchain data, making queries faster and more efficient. Then, users can query indexed data using GraphQL.

The Graph Network serves as an inspiration for this thesis; however, our approach focuses on building a centralized data pipeline instead of a decentralized indexing system.

Chapter 4

Analysis

Blockchain data is complex, constantly growing, and requires efficient processing for meaningful analysis. This chapter examines the key challenges in handling blockchain data and explores the need for a robust data pipeline. We introduce our selected use case, MEV Analysis, to ensure that the pipeline meets real-world research needs. Additionally, we analyze existing solutions and highlight their limitations. Finally, we define the functional and non-functional requirements of our system and provide a technical breakdown of our architectural decisions.

4.1 Introduction to Use Case

In our work, we adopt a use-case-centric approach for the analysis and design of the production-ready data pipeline. The reason for this decision is that by focusing on a real-world use case, we can come up with more realistic requirements and we can design the pipeline with the practical needs of potential users, specifically researchers working on blockchain data analytics. In this way, we can ensure that the system is not only technically robust but also relevant and intuitive for real-world applications.

For this purpose, we selected Maximal Extractable Value (MEV) analysis as our representative use case.

4.1.1 Overview of MEV Analysis

MEV refers to the maximum profit that can be obtained by manipulating the order, inclusion, or exclusion of transactions within a block. In simple terms, when transactions are waiting to be added to the blockchain, the entity responsible for block creation (such as a miner or validator) can reorder these transactions to extract profit. This profit is

derived from opportunities like arbitrage, front-running, and sandwich attacks, where certain transactions become more valuable when placed strategically before or after the other transactions [43]

For example, in a sandwich attack, a malicious actor places one transaction just before a transaction in which the malicious actor's transaction buys an asset and then puts another one just after it, and this one sells the asset at a higher price. In this way, they profit from the price impact caused by the target transaction [44]. Another common scenario is arbitrage, where traders exploit price differences across decentralized exchanges (DEXs) by buying assets cheaply on one platform and selling them at a higher price on another [45].

4.1.2 Importance and Relevance of MEV Analysis

MEV analysis is an ideal use case for evaluating the effectiveness of our pipeline because it has complex data processing requirements, requires data from both on-chain and off-chain data, and also gives insightful information about the economics of blockchain.

In the MEV analysis, understanding the dependencies between transactions is important, particularly how reordering impacts profitability. Analyzing these complex interactions requires a careful selection of data sources and the ability to efficiently join and process them. In addition, MEV opportunities often arise when on-chain blockchain data such as transaction logs, is combined with off-chain market data such as price information from centralized exchanges and this requires the pipeline to support heterogeneous data ingestion [46]. Finally, MEV activities influence the economic incentives of miners and validators, which affects gas prices and transaction prioritization, and understanding these economic impacts allows for a more realistic protocol design [45]

4.1.3 Key Analytics for MEV Analysis

We focus on two critical analyses of MEV: Price Impact Analysis and Gas Price and Transaction Priority Trends Analysis.

In Price Impact Analysis, we want to learn how market prices on Decentralized Exchanges (DEXs) and Centralized Exchanges (CEXs) influence MEV strategies by finding high-value transactions and analyzing their impacts on the price movements and how these movements are relevant to gas fees and transaction sizes. Similarly, in Gas Price and Priority Trends Analysis, we want to learn how strategies such as front-running and sandwich attacks affect gas prices and transaction prioritization by analyzing gas price fluctuations during these events and identifying transaction ordering patterns.

4.2 Technical Causes

In this section, technical causes why we need a robust, production-ready data pipeline for blockchain are discussed.

4.2.1 Complexity of Blockchain Data

Analyzing blockchain data presents significant technical challenges due to its complexity, volume, heterogeneity, and evolving nature.

Blockchain networks like Ethereum generate millions of transactions, event logs, and blocks, resulting in terabytes of on-chain data over time. In addition, there are off-chain data like mempool data, and P2P network data which make total data size even higher (see Section 2.1.3), and this data is generated in near real-time requiring the pipeline to support low-latency ingestion and real-time processing to be able to ingest successfully.

Moreover, all blockchain data is not coming only from one source but rather from multiple sources such as full nodes, CEXs, P2P network crawlers, etc., resulting in different data formats so a pipeline should be flexible to gather all different kinds of data formats and efficiently integrate and store them.

Another important consideration of blockchain data is its dynamic and evolving nature. They regularly undergo protocol upgrades (e.g. Ethereum's Merge, hard forks [47]) affecting their data schemas and API structures. These changes can break existing data pipelines if they don't have a flexible architecture and storage mechanism.

4.2.2 Limited Existing Solutions

While several tools and approaches exist for blockchain data processing, they each exhibit some limitations that prevent them from supporting the complex requirements, which will be discussed in Section 4.3 in more detail. In this section, these approaches and their weak points are analyzed.

Data Collection Pipeline for Blockchain Platforms by Ilteber Ayvaci

As mentioned in Section 3.1.1, Ayvaci's thesis [41] is one of the foundation works we use while developing our pipeline. However, it has some bottlenecks and weak points that need to be covered to be able to develop a production-ready data pipeline that can ingest large-scale data.

Ayvaci's work doesn't follow a formal data pipeline architecture such as ETL or ELT, which are essential for the extensibility, maintainability, and scalability of data pro-

cessing workflows. Moreover, it does not use industry-standard tools for data pipelines such as data lake tools (e.g. minIO), query engine tools (e.g. Apache Spark, DuckDB), and monitoring tools (Prometheus, Loki, Grafana). As a result, these absences lead to complex implementation and a highly custom codebase that is difficult to maintain, more prone to bugs, and harder to onboard for new developers.

In addition, due to its main focus on creating a data collection pipeline for blockchain platforms rather than a production-ready system that can be served to users, it lacks software components such as integration with client-server architecture or with monitoring systems, which makes it not possible to be used by multiple users supported by powerful servers, instead, it runs locally on the user's machine.

TUMChainBook

TUMChainBook is a multi-user Jupyter Notebook platform with custom frontend and processing service components as mentioned in Section 3.1.2. It is the other foundation work we use in our pipeline.

Similarly to Ayvaci's work, TUMChainBook is not designed to be used in a production-ready system. Although it uses an established JupyterHub solution that is developed for production use and backed by Kubernetes [42], it has a primitive storage and processing service to store and ingest blockchain data, which is developed for demo purposes.

Its processing service fetches only on-chain data and it fetches daily with cron jobs instead of fetching them in real-time and stores fetched data in a shared volume as CSV files. Therefore, it lacks real-time data ingestion and scalable storage and processing capabilities necessary for handling complex blockchain data.

RPC Nodes

RPC nodes are blockchain nodes that directly talk with the blockchain network to gather data and have APIs to serve them. There are different implementations of them for different use cases and workloads [48]–[50]. However, their primary focus is to store on-chain information like transaction details, block data, and smart contract states and they don't store off-chain data such as mempool data instead they just serve them in real-time. They are great to be used as a data source to fetch on-chain and off-chain data, but they alone don't meet our requirements.

Dune Analytics

Dune Analytics [1] is a community-backed data platform that provides a user-friendly interface for querying blockchain data using SQL, as mentioned in Section 3.2.2. It

is one of the most used platforms for blockchain research. However, it also has some limitations.

It has community-driven data models so users rely on predefined data models to be worked on, which limits the customization required for specialized analyses. It also has restricted processing flexibility and does not support the custom data transformation we need for our heterogeneous workload.

4.3 Requirements Analysis

4.3.1 Functional Requirements

The functional requirements define the core capabilities that the data pipeline must provide to handle blockchain data processing efficiently and ensure a user-friendly experience for researchers.

Data Ingestion

- *Continuous, Real-Time Data Ingestion:* The pipeline must support continuous, real-time data ingestion (see Section 4.2.1).
- *Multi-Source Data Support:* The pipeline should be able to ingest data from multiple blockchain sources, including on-chain and off-chain data
- *Dynamic Schema Handling:* The pipeline must be able to adapt to changes in blockchain data schemas (see Section 4.2.1) without manual intervention.

Data Access

- *Flexible Querying:* Researchers should be able to perform queries using SQL-like interfaces for ad-hoc data exploration
- *Cache Support:* Researchers should be able to persist their query results in the system for later, efficient access.
- *User-Friendly Interface:* The system should provide a user-friendly user interface for researchers to access the system and run their queries.

User Management

- *Authentication and Authorization:* The system should implement an authentication system that securely stores user data and authorizes valid access.

Monitoring

- *Centralized Monitoring:* The system should provide a centralized place with dashboards tracking various metrics obtained from the pipeline.
- *Centralized Logging:* The system should provide a centralized place for logs to be stored and easily accessed by admins.

4.3.2 Non-Functional Requirements

The non-functional requirements define the quality attributes that the production-ready data pipeline must exhibit. Below are the key non-functional requirements that the proposed pipeline must fulfill:

1. *Scalability:* The pipeline must efficiently handle high volumes of blockchain data and support growing workloads by scaling storage, processing, and data ingestion components as needed.
2. *Simplicity and Maintainability:* The system should have a clean and modular architecture with readable, well-documented code, allowing for easy maintenance and debugging.
3. *Performance Efficiency:* The pipeline must achieve high throughput and low-latency data processing while optimizing the use of computational and storage resources, ensuring cost-effective operations without compromising performance.
4. *Extensibility:* The pipeline must be designed to easily integrate new data sources, blockchain networks, and changes to data schemas without major architectural changes.
5. *Observability:* The system should provide real-time visibility into system performance, data ingestion rates, and processing workflows.
6. *Security:* The system should enforce secure communication protocols for data in transit and minimize the attack surface through firewall configurations and secure API endpoints.

4.4 Technical Analysis

This section provides an in-depth technical analysis of the decisions made for the proposed pipeline.

4.4.1 Blockchain-Specific Analysis

This section examines key blockchain-related technical choices, including Ethereum RPC clients and node configurations, to ensure optimal data ingestion and processing within our pipeline.

Ethereum RPC Clients

RPC Clients enable the pipeline to interact with Ethereum nodes by providing APIs to fetch blockchain data (see Section 3.2.1). There are many different implementations for different workloads and requirements. We choose Erigon [48] over the others because it has an active community and is actively maintained, ensuring possible bugs we can face can easily be resolved. In addition, it uses multi-threaded architecture and advanced data compression techniques, which make it an ideal candidate for workloads requiring faster data retrieval and reduced resource consumption. This efficiency allows us to allocate more computational resources to the pipeline's core processing tasks rather than data collection, which is particularly important given our goal of serving multiple users simultaneously through the JupyterHub infrastructure [42].

Ethereum Node Configurations

There are different configuration options for Ethereum nodes such as defining a node type as archive node, full node, and light node [51]. We decided to use a full node since it gives a good balance of resource usage and meets our requirements.

A full node has access to an entire blockchain state and real-time off-chain data, such as transaction mempool, unlike archive nodes, which also store historical state data but have higher resource consumption. On the other hand, a light node has lesser resource consumption but is insufficient for MEV-related analytics that require real-time mempool access. As a result, a full node implementation meets our requirements such as providing us real-time data and multi-source data support and it also avoids unnecessary complexity, resulting in a simpler and maintainable data source (see Section 4.3.2).

4.4.2 Data Storage and Processing Analysis

Efficient data storage and processing are critical for building a scalable blockchain data pipeline. This section examines key architectural decisions, including the choice of a data lake over alternative storage models, data ingestion strategies, storage solutions, and transformation tools. Each decision is evaluated based on scalability, maintainability, and performance considerations to ensure the pipeline meets its functional and non-functional requirements.

Data Architectures: Why Data Lake?

Three different data architectures can be used in our pipeline, discussed in Section 2.2.1: data lake-based, data warehouse-based, and hybrid. Using data lake-based architecture with ELT processing model addresses our requirements (see Section 4.3) and workload more because of the following reasons:

Advantages over Data Warehouse-based Architecture

Schema Flexibility: Data lakes are designed with schema-on-read architecture, allowing data to be ingested in its raw form and structured only during querying. This approach supports dynamic schema evaluation of blockchain systems (see Section 4.2.1 and Section 4.3.1). Thanks to its schema-on-read design, we don't need to do manual migrations for changes in blockchain data schema or when we want to add new data sources, which gives us great flexibility and allows us to extend our pipeline and easily maintain it (see Section 4.3.2).

Decoupled Storage and Compute: Data lake-based architecture allows us to deploy storage-heavy components and compute-heavy components to their respective specialized nodes. For example, we can deploy our data lake to a node that has high storage capacity but low computing power and our query engine to a node that has low storage capacity but high computing power. Additionally, we can scale storage and computing independently since they are not coupled to each other. Thus, we can meet our scalability requirement (see Section 4.3.2) thanks to this architecture choice.

Designed for Researchers: Data lakes provide direct access to raw, unstructured data, which is crucial for researchers working with blockchain data since they want to do complex data exploration and prefer flexible access, since they cannot know beforehand what kind of data their research exactly requires. Thus, having a flexible architecture aligns well with the requirements of our user base.

Advantages over Hybrid Architecture

Reduced System Complexity: While hybrid models combine the benefits of both lakes and warehouses, they introduce higher complexity (see Section 2.2.1). Data lakes offer simplicity by eliminating the need for complex data management, and this results in a simpler and more maintainable architecture that supports our non-functional requirements (see Section 4.3.2).

Resource-Efficiency: Hybrid models often require maintaining two separate systems, increasing both operational cost and resource usage. Therefore, it is not resource-efficient and maintainable for our workload.

Data Collection: Why Cron Jobs and Always-Running Pods?
To meet our requirements (see Section 4.3), we need a data collection strategy that ensures continuous, reliable ingestion while balancing resource efficiency, simplicity, and performance. We evaluated three common data ingestion methods: event-driven ingestion (Kafka), scheduled ingestion (cron jobs), and continuous listening (always-running pods). After our analysis, cron jobs and always-running pods are selected as the most suitable approaches. The following arguments detail the justification for these choices:

Event-driven ingestion (Kafka): Apache Kafka is a popular event-driven ingestion tool [52], but we decided not to use it because it adds unnecessary architectural complexity to our pipeline by introducing additional layers (e.g. brokers, producers, consumers). Since we already using a data lake architecture, raw data can be directly ingested into storage without the need for a distributed streaming platform. We can basically buffer collected data in memory in our services and then save it in batches to our data lake without requiring additional complexity. In this way, we can ensure we both spend our development time and resource usage efficiently to meet our performance efficiency requirement (see Section 4.3.2).

Scheduled Ingestion (Cron Jobs): Cron jobs provide a simple and effective solution for tasks that need to run at regular intervals because they run only when needed, making them highly resource-efficient by avoiding idle resource consumption. Thus, they are chosen because of their resource efficiency and simplicity (see Section 4.3.2).

Continuous Listening (Always-Running Pods): Always-running pods are critical for real-time data collection where a persistent connection is required. They can keep persistent connections to data sources with minimal latency by using streaming protocols such as WebSockets (see Section 2.3.1). Even though they require more resources, they are necessary to cover our continuous and real-time data ingestion requirement (see Section 4.3.1).

Data Storage: Why MinIO?
After selecting a data lake architecture for the pipeline (see Section 4.4.2), the next critical decision is choosing the storage backend that best aligns with our requirements (see Section 4.3). There are three primary options: local filesystem, HDFS, and MinIO. After careful analysis, MinIO is selected as the preferred data storage solution for the following reasons:

Advantages over Local Filesystem

The local filesystem offers simplicity but lacks scalability, flexibility, and production-grade features.

Advanced Functionality Out-of-the-Box: MinIO provides functionalities such as automatic data replication, lifecycle management, and object retention expiration, which are critical for our pipeline that a local filesystem cannot natively support.

Scalability and High Availability: A local filesystem is inherently bound to single-machine storage limits, making horizontal scaling difficult. On the contrary, MinIO supports horizontal scaling thanks to its distributed nature, and it replicates data over its distributed storage, which makes it possible to ensure higher availability than the local filesystem, since even if one deployment fails, the other one can still continue to serve. Therefore, it meets the scalability requirement of the pipeline (see Section 4.3.2).

Advantages over Hadoop Distributed File System

While HDFS has been a traditional choice for big data applications, its architecture and operational model are less suited for modern, cloud-native environments like Kubernetes (see Section 2.2.2).

Cloud-Native Integration: MinIO has a built-in integration with Kubernetes, which makes it possible to deploy it easily to a cloud-native environment. On the other hand, HDFS is not inherently compatible with Kubernetes, and it is built for clusters where storage and computing are closely linked, which makes it incompatible with cloud-native environments and requires manual implementations. Therefore, it is not simple and maintainable to use HDFS as our data lake solution.

Built-in Monitoring Support: MinIO also has built-in monitoring support that can directly expose Prometheus metrics and can be easily monitored. Therefore, it improves the overall observability of the system without adding additional complexity, and it meets our observability and simplicity requirements (see Section 4.3.2).

Data Transformation: Why DuckDB?

For the data transformation, we considered two primary options: DuckDB and Apache Spark. Both tools offer SQL-like interfaces and S3-compatible access patterns to access the MinIO data lake. Therefore, it is easy to switch from one to another.

However, they differ in terms of scalability and underlying architecture. Apache Spark is a distributed processing engine designed for large-scale, parallel data processing. It is highly scalable, resource-intensive, and not easy to configure (see Section 2.2.3). In contrast, DuckDB is an in-process stateless query engine that runs locally in the deployment where it is initialized. Although it is not inherently distributed, it is stateless, which means we can achieve horizontal scalability by running multiple instances of it simultaneously.

Because they are easy to switch, we follow an iterative development approach for this decision. We aim to build a simple, end-to-end pipeline first by using DuckDB since it is easy to configure. Then, if we see any performance bottlenecks while evaluating our system, we can easily switch to Apache Spark which gives us better performance promises. In this way, we aim to keep our pipeline simple, maintainable, and resource-efficient until it is necessary to make it more advanced.

Chapter 5

Design

The Design chapter outlines the architecture and key components of the blockchain data pipeline. It explains how the system is structured to ensure scalability, flexibility, and maintainability while efficiently handling blockchain data. The pipeline follows a modular design with distinct layers for data extraction, storage, transformation, and access. Additionally, a monitoring layer is integrated to track system performance and ensure reliability. This chapter provides an in-depth look at each layer, explaining its roles, interactions, and the technologies used to implement them.

5.1 Architectural Overview

The system architecture is composed of six core components, each responsible for specific tasks in data ingestion, processing, storage, and access. The architecture is modular, allowing each component to scale independently while ensuring resource optimization and high availability. A simplified diagram showing the six core components and their relations can be seen in Figure 5.1. The basic idea of the architecture is to process data step by step with a straightforward design and finally serve it to the TUMChainBook (see Section 3.1.2) that acts as our frontend component. Below is a brief overview of the key architectural components of our system:

Data Sources: The pipeline ingests data from multiple blockchain-related sources, including on-chain and off-chain data. The data sources are selected according to the needs of the MEV use case (see Section 4.1).

Data Fetchers: These are lightweight microservices responsible for extracting data from the data sources. Each of them focuses on fetching data from a single data source,

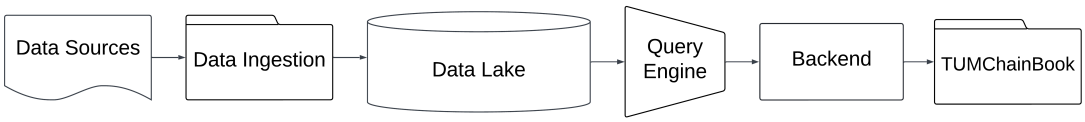


Figure 5.1: Simplified Architecture

aiming to decouple fetchers from each other. They perform basic data validation, format normalization, and finally, the save operation to the data lake.

Data Lake: It acts as the central storage layer, providing flexible, scalable storage for raw, unstructured data. It supports the schema-on-read approach, allowing data transformations and schema definitions to occur during querying, not at the time of ingestion.

Query Engine: On-demand data transformation and query processing happen here. It executes analytical queries directly on the data stored in the data lake and provides a user-friendly, SQL-like interface.

Backend: It serves as the intermediary layer between the query engine and the TUM-ChainBook, facilitating secure communication, query execution management, and user access control.

TUMChainBook: It acts like our frontend, but it does not consist only of a user interface instead, it is a customized complex JupyterHub system (see Section 3.1.2). It provides a computing environment for users to run their notebooks and access the data pipeline via the backend.

The overall design focuses on simplicity, maintainability, and scalability, ensuring that each component can be independently scaled and maintained. By adopting an ELT model, the pipeline supports flexible data processing, with schema definitions applied on-demand during querying.

5.2 Layers

To ensure scalability, flexibility, and maintainability the pipeline is structured using a customized ELT pipeline model. This layered approach breaks the system into distinct functional units, each responsible for a specific aspect of the processing flow.

Unlike traditional ELT, where transformation occurs after loading data, our customized pipeline model performs on-demand transformations during data access and optionally ability to persist the results for later access. This allows flexible access and transfor-

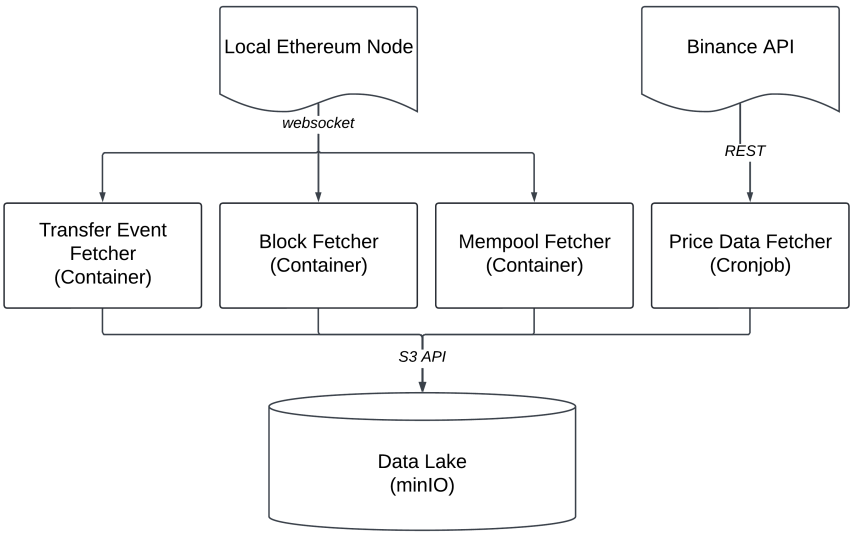


Figure 5.2: Architecture: Extract and Load Layers

mation of raw data, which is needed by our user base, which consists of researchers. Additionally, a monitoring layer is integrated to ensure the observability and health tracking of all pipeline components.

5.2.1 Extract Layer

The Extract Layer serves as the entry point of the data pipeline, responsible for collecting raw data from relevant blockchain-related sources and loading it into the data lake for further processing. It consists of two parts: data sources and data fetchers. Figure 5.2 illustrates the components and how they interact with each other.

Data Sources

The Extract Layer ingests data from two primary sources, critical for our use case MEV analysis.

Local Ethereum Node: It provides access to on-chain and off-chain data, including transfer event logs, block data, transaction data, and mempool data. The access is handled via WebSockets for low-latency, real-time communication. The local Ethereum node is chosen as our main access to Ethereum data rather than public nodes because we require a high volume of blockchain data from this source and third-party node providers are more expensive than running a node locally. In addition, they provide limited data so it is hard to collect off-chain data directly from them. Finally, by running our local node, we ensure full control over data without exposing our usage patterns to external

providers. In this way, we ensure that we meet performance efficiency and security aspects of our requirements (see Section 4.3.2).

Binance API: Binance is one example of CEXs and provides off-chain price data, critical for our MEV use case, particularly for price impact and arbitrage detection analyses. It serves its data by exposing REST APIs. We chose Binance as our primary price data source since it is popular among people and has a high volume of usage.

Data Fetchers

Data ingestion is handled by two different types of specialized fetchers: always-running pods and cron jobs to ensure real-time data access and resource efficiency, respectively (see Section 4.4.2). The fetchers are running independently from each other, ensuring there is no coupling. Thanks to its decoupling, we can easily extend our pipeline with new data sources and fetchers simply by adding additional fetchers. This helps us to meet our extensibility requirement (see Section 4.3.2).

There are currently four fetchers implemented in total:

1. Transfer Event Fetcher (Always-running pod): It monitors and fetches transfer events from the local Ethereum node in real-time via a WebSocket connection, ensuring low-latency event capture.
2. Block Fetcher (Always-running pod): It continuously fetches block data as new blocks are added to the blockchain via a WebSocket connection.
3. Mempool Fetcher (Always-running pod): It retrieves pending transactions from the mempool via a WebSocket connection.
4. Price Data Fetcher (Cron job): It periodically collects price data from the Binance API via its REST APIs.

Each of them primarily fetches, validates, and normalizes data and then buffers it in memory and when there are enough records, it saves them to the MinIO data lake in batches by using its S3-compatible APIs. If the data lake is temporarily unavailable, data remains in memory until successful storage is confirmed. In case both the data fetchers and data lake are simultaneously unavailable, data loss is accepted as a trade-off to maintain a less complex, easily maintainable architecture.

5.2.2 Load Layer

The Load Layer plays a critical role in the pipeline by providing a scalable, flexible, and efficient storage solution for the raw data collected by the Extract Layer. In our customized ETL pipeline model, the Load Layer ensures that unstructured raw data is

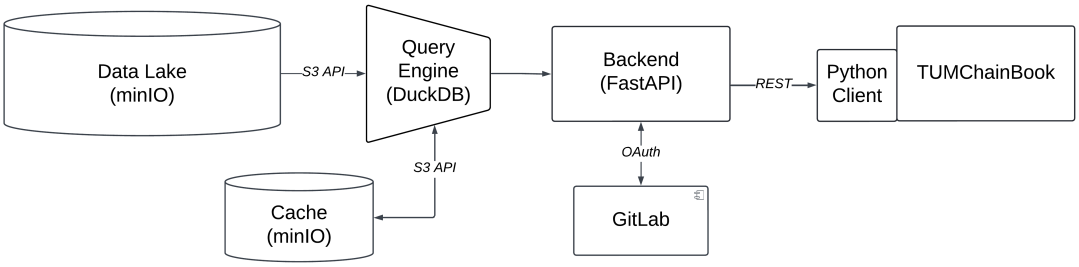


Figure 5.3: Architecture: Transformation and Access Layers

securely stored and available for on-demand transformations in the Access and Transform Layer. The Load Layer consists of two key components: storage model (object storage with MinIO) and data format (optimized storage with Parquet).

Storage Model

The load layer stores raw data with minimal preprocessing performed during the ingestion phase using its flexible data lake solution. As it is explained in Section 4.4.2, MinIO is the data lake solution thanks to its scalability, high availability, out-of-the-box advanced functionalities, and cloud-native integration. By choosing MinIO as our data storage solution, we cover the scalability and maintainability requirements of our system (see Section 4.3.2). It has a distributed architecture that supports horizontal scaling by adding new storage instances and distributing the load. It is also an established solution with its mature ecosystem, extensive documentation, and active community support, which simplifies maintenance and future upgrades of the overall system.

Data Format

We mainly use Parquet as our data format for our unstructured data because of its performance, efficiency, and seamless integration. It uses a columnar storage format, which is highly optimized for analytical queries, allowing for faster read performance compared to row-based formats like CSV or JSON. In addition, thanks to its columnar nature, it efficiently compresses the data, resulting in a significant reduction in the storage requirements. Finally, Parquet is supported by various data processing frameworks and query engines (Apache Spark, DuckDB) by default, making its integration with these systems a lot easier.

5.2.3 Transformation and Access Layer

The transformation and access layer serves as the interface between the data pipeline and end users, providing secure, scalable, and flexible access to the stored data in the

data lake. This layer handles on-demand data transformations, query execution, and user management through a backend service. It consists of user access, caching, and user management components. Figure 5.3 illustrates the components and how they interact with each other.

User Access

The user access component ensures that researchers can query and analyze data with minimal latency while maintaining flexibility and usability.

Query Engine: DuckDB

DuckDB serves as a query engine that directly queries data stored in the MinIO data lake using S3-compatible APIs. Its SQL-like interface allows researchers to query Parquet files without pre-loading data into a separate database, offering a user-friendly querying experience. DuckDB performs in-memory computations, ensuring low-latency querying and thanks to its lightweight structure, it minimizes resource usage and thanks to its stateless architecture, it allows for horizontal scaling by deploying multiple DuckDB instances, working independently (see Section 4.4.2).

Backend

The backend is the main contact point of the user requests and it connects other components of the layer to process the requests. It exposes a RESTful API that allows secure access to data through the DuckDB query engine. It queries DuckDB and serves the results of DuckDB as Parquet files to users to ensure efficient data transfer through Parquet's data compression.

TUMChainBook is the frontend component of our system and users use it to send requests to the backend. To fully integrate TUMChainBook and the backend and provide a user-friendly interface, a Python client is used that abstracts the REST API calls and authentication and provides users functions they can call to run their requests.

The backend components also don't store states so they can also be horizontally scaled by running multiple instances to handle higher request loads.

Caching (Transformation)

The caching component introduces a persistent query system, allowing users to store transformation results for faster future access.

The idea of the persistent queries is that researchers can save the results of frequently executed queries in a fast-access storage layer and perform the transformation step by

end-users on the fly because researchers (end-users) often cannot define all transformation requirements priorly because of the nature of doing research. Therefore, by using persistent queries, they can do the transformation they want and store the results in the storage that only they can access, which allows greater flexibility for the transformation step.

The idea of persistent queries suits our user base better than providing a data warehouse with pre-defined transformations, thanks to its flexibility. At the same time, providing such a data warehouse would introduce additional complexity and need for maintenance, which conflicts with our requirements (see Section 4.3.2).

The persistent query system also reduces the load on the pipeline by reducing computation duplication. At the same time, the persistent queries have retention periods that delete the results after a defined time to avoid increasing storage needs and result in a system that is more performance efficient in terms of both computation and storage.

Authentication and User Management

The backend uses the OAuth framework [29] for secure access to the system via the GitLab identity provider. In this way, the backend itself doesn't have to store user data or handle the authentication; rather, a well-known GitLab identity provider does it. To ensure further security, all API communications are TLS-encrypted, ensuring data privacy and secure interactions.

There is also a role-based access control in the backend with two assigned roles: admin and researchers, each with specific access permissions, and their members are controlled via GitLab groups. Admin users can define longer retention periods and run their queries longer than researchers allowing them to do more complex computations.

5.2.4 Monitoring Layer

The monitoring layer plays a supportive yet essential role in ensuring that the data pipeline operates smoothly. While this layer does not directly contribute to data processing and storage, it provides critical insights into the system's health, performance, and stability. It consists of three main components: metrics collection, log aggregation, and visualization. Figure 5.4 illustrates the components and how they interact with each other

Metrics Collection

Metrics provide quantifiable insights about the performance and resource utilization of the pipeline. Tracking these metrics allows for proactive issue detection and also performance tuning. Prometheus is used as the metric collection and storage solution

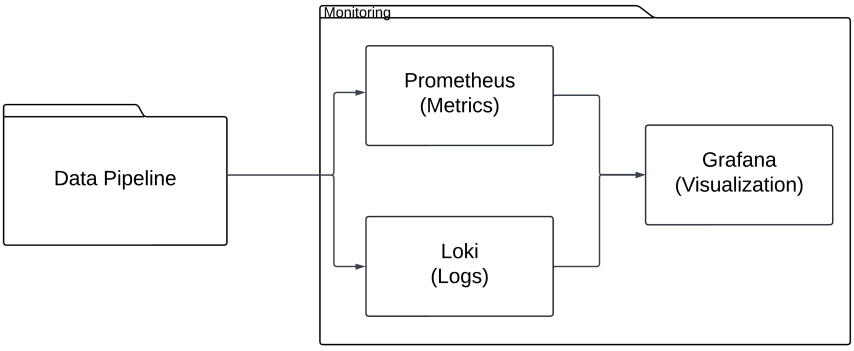


Figure 5.4: Architecture: Monitoring Layer

because it is a popular and established solution with a big user base and community support (see Section 2.3.3). In addition, it is a lightweight monitoring solution that is easy to configure and less resource-intensive.

We collect two types of metrics using Prometheus:

1. Application-specific metrics such as data ingestion volume, query latency, etc.
2. Machine-specific metrics such as CPU usage, memory consumption, network I/O, etc.

Log Aggregation

Logs provide contextual information about system events, which is essential for troubleshooting, debugging, and security auditing. Centralized logging ensures that logs are retained even if individual containers fail, allowing for continuous monitoring. Fluentd is used as the log collector and forwarder, crawling logs from all services of the pipeline. It, then, aggregates logs and forwards them to Loki. Loki is used as the centralized location of the logs and it can easily be queried for the log access.

Visualization

Visualization provides a clear and intuitive representation of complex metrics and log data, making it easier to identify trends, detect anomalies, and track performance. Grafana is used as a unified visualization solution because of its easy integration with Loki and Prometheus and also big community support. It helps us to create dynamic dashboards that display real-time metrics and log information. In addition to observing the system's health, Grafana is also used to visualize workload emulations, tracking key evaluation metrics of the system so that we can detect the bottlenecks of the pipeline and proactively solve them.

Key Benefits of the Monitoring Layer

The Prometheus + Loki + Grafana stack is designed for efficiency, ensuring that monitoring activities do not consume significant system resources. Given that monitoring is a supportive component, the lightweight nature of this stack ensures that it does not interfere with the primary functionalities of the pipeline.

The monitoring layer provides real-time visibility of the system, allowing one to observe what is happening inside the system. It helps us to proactively solve problems and meet the respective requirements of the system (see Section 4.3.2). Moreover, thanks to this visibility, the overall maintainability of the system is increased since developers can debug the issues and solve them easily with the monitoring layer.

Chapter 6

Implementation

This chapter explains how we built and deployed our blockchain data pipeline. It covers the code structure, infrastructure setup, and how different services work together to ensure scalability, reliability, and efficiency. We also describe our step-by-step development process, showing how the system improved over multiple versions to become a fully functional and production-ready solution.

6.1 Code Structure

The codebase is structured into three primary parts: infrastructure, modules, and services. A visual representation of the folder structure is included in Figure 6.1.

6.1.1 Infrastructure

This folder is responsible for setting up the infrastructure, deploying, and managing all services in a containerized environment (see Section 6.2 for more information).

6.1.2 Modules

The modules folder contains the core logic of the pipeline, broken into three key areas: data access, data ingestion, and data storage. These modules ensure a separation of concerns, making the system modular and extensible (see Section 6.3 for more information).

6.1.3 Services

The services directory is responsible for initializing all classes, constructing their dependencies, and passing environment variables. It also contains the container configuration

of each service. By separating initialization logic from class definitions, the design follows the principle of dependency inversion, which makes the dependency management easier.

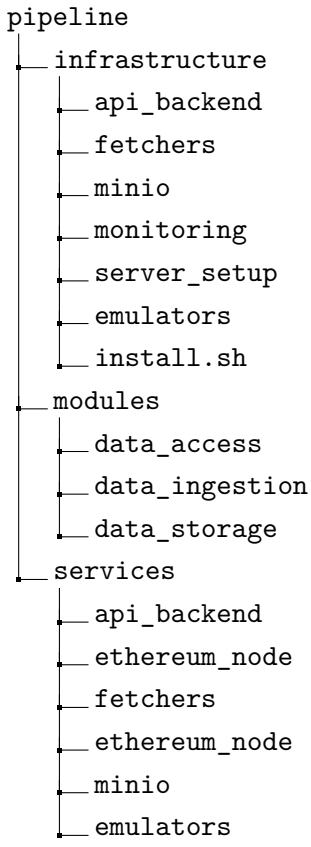


Figure 6.1: Pipeline Folder Structure

6.2 Infrastructure

This section describes the node deployments, detailing how each node is configured and its specific role within the pipeline. It then covers service deployments, explaining how different components are deployed using Kubernetes. Lastly, it discusses external access to the pipeline, outlining networking and security mechanisms.

6.2.1 Node Deployments

The deployment of our system is structured across multiple physical nodes, each dedicated to a specific role to optimize performance, scalability, and maintainability. We use the physical, on-premise machines for our deployment. Each node has a unique name and distinct specifications, including variations in storage capacity, computing power,

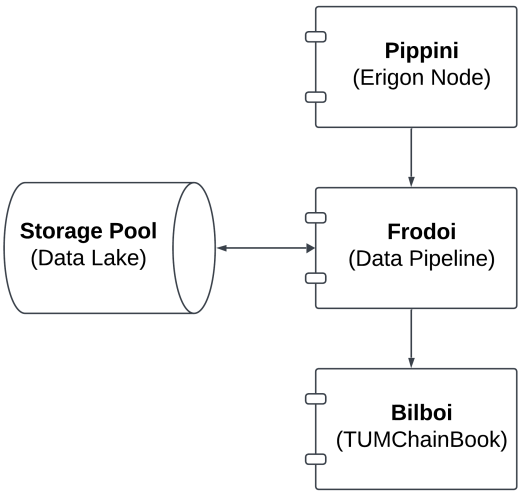


Figure 6.2: Physical Node Deployments

and network access. These differences require a strategic placement of services, ensuring that each component of the pipeline is deployed in an environment best suited for its performance needs.

The main design consideration is separating compute and storage nodes to ensure that workloads are efficiently distributed, allowing resource separation between compute-intensive processes and storage-heavy operations. Figure 6.2 illustrates the deployments of the physical nodes.

Pippini Node (Ethereum Local Node: Erigon)
Pippini node has the following specifications:

1. 8TB SSD Storage
2. High-Computing Power (Intel(R) Xeon(R) D-2123IT CPU @ 2.20GHz)
3. High-speed Internal Network (1Gbit)

The main responsibility of Pippini is to run Erigon, a high-performance Ethereum full-node client. It is our primary data source for blockchain data, including on-chain and off-chain data such as blocks, transactions, logs, and mempool data.

Pippini is chosen to run the Erigon node because blockchain nodes require high storage capacity and fast disk access. It also has high computational power to run block operations, sync with other blockchain nodes, and query blockchain data. Pippini can meet these requirements with its 8TB SSD storage and high-computing power.

Frodoi Node (Data Pipeline)

Frodoi node has the following specifications:

1. 8TB SSD Storage
2. High-Computing Power (Intel(R) Xeon(R) D-2123IT CPU @ 2.20GHz)
3. High-speed Internal and External Network (1 Gbit)
4. Public IP

It serves as the core processing unit of the data pipeline and is responsible for data ingestion, storage, and query execution. It acts as the central bridge between different system components, facilitating smooth data flow and ensuring efficient interaction across services.

Frodoi communicates with the Erigon node running on Pippini to fetch real-time on-chain and off-chain data. It also retrieves additional off-chain data from external APIs. Then, it pre-processes ingested data and stores it in the Storage Pool. Finally, it exposes a REST API and runs the requests coming from TUMChainBook, which are the analytical queries done to the system to perform on-the-fly transformations.

The data pipeline requires high computing power since it involves real-time ingestion of blockchain data and aggregation, filtering, and transformation of the collected data. It also requires a high internet speed since it continuously fetches data from the Erigon node and serves the collected data to TUMChainBook. Additionally, it also requires a public IP since TUMChainBook runs externally and needs to interact with the data pipeline. Finally, it requires high storage since it caches and buffers data ingested and served. All of these requirements are met by Frodoi since it has high storage space, high computing power, a high-speed network, and a public IP.

Storage Pool (Data Lake)

The Storage Pool has the following specifications:

1. 40TB HDD Storage
2. Low-Computing Power
3. High-speed Internal Network (1 Gbit)

It runs the Network File System to provide a shared storage system that allows efficient access to stored data from compute nodes. The main responsibility of the storage pool is to act as the centralized data repository for the pipeline for efficient long-term storage.

It stores all ingested data and doesn't perform any computation. It only stores and serves the files over the network to the pipeline.

The Storage Pool is a good choice to store all the ingested data since it uses cost-efficient HDD storage and it has a high-speed internal network to store and serve the pipeline data. In addition, it has low computing power but doesn't perform any computation.

Bilboi Node (TumChainBook)
Bilboi node has the following specifications:

1. 200GB SSD Storage
2. High-Computing Power (Intel(R) Xeon(R) D-2123IT CPU @ 2.20GHz)
3. High-speed External Network (1 Gbit)
4. Public IP

It runs TUMChainBook, a Jupyter-based research platform for blockchain analytics, and provides an interactive interface for researchers to explore blockchain data. TUMChainBook requires minimal storage, as storage and computations are offloaded to the data pipeline, and it only stores the results of the queries. However, it still requires high computing power since it runs the notebooks of the users and it is a multi-user platform, meaning multiple user can perform their computations at the same time. It also requires a public IP for users to access its interface.

Bilboi is a good choice for running TUMChainBook since it has low but fast storage and high-computing power, as well as a public IP.

6.2.2 Service Deployments

The service deployment strategy of our system is centered around containerization and automatic orchestration of the services, ensuring scalable, resilient, and maintainable deployments. To achieve this, we use Kubernetes, K3s, Docker, and Helm.

Kubernetes is responsible for managing and scaling containerized applications across our infrastructure [53]. It ensures that services are automatically restarted in case of failure, dynamically adjusts resource allocation, and handles networking and service discovery between components. Instead of using the full Kubernetes distribution, we deploy K3s, a lightweight, production-ready Kubernetes distribution designed for resource-efficient environments [54]. All services in our system are containerized using Docker, which provides a consistent and isolated execution environment for our services [55]. Finally, we

use Helm, a package manager for Kubernetes, to automate and simplify the deployment of complex applications [56].

Fetchers

The fetchers are implemented using two different approaches depending on the nature of the data source: always-running containers and scheduled jobs. Fetchers that require a persistent connection to real-time data, such as mempool transactions, run as Kubernetes deployments, ensuring continuous and reliable data collection thanks to the auto-recovery feature of Kubernetes. On the other hand, fetchers that need to retrieve data periodically, such as historical price data, run as Kubernetes cron jobs, executing at scheduled intervals to collect and store new data efficiently.

To ensure secure and efficient communication, fetchers interact with their respective data sources through optimized network configurations. For Ethereum on-chain data, fetchers connect to the local Erigon node hosted on the Pippini node via internal network communication, reducing latency and avoiding reliance on external APIs. For price data from Binance, fetchers use external network access to retrieve market data through the Binance API. Once data is collected, fetchers store it in MinIO data lake via internal Kubernetes networking, ensuring fast and secure data transfer.

Backend

The backend is deployed as a Kubernetes deployment, using a Docker container built with the backend code. The deployment resource allows us to run multiple instances of the backend in parallel, ensuring that incoming requests are distributed efficiently. Since our backend is stateless, each instance operates independently, meaning we can easily scale it by adding or removing instances based on demand. Kubernetes automatically manages this scaling, balancing the load across available containers and restarting any failed instances to maintain system reliability.

MinIO

We deploy two separate MinIO instances using Helm charts, each serving a different purpose.

The first MinIO instance acts os our data lake, running on the Frodoi node, but storing data in the Storage Pool, which is mounted via NFS. This allows Frodoi to handle data ingestion and processing while efficiently storing large volumes of raw data.

The second MinIO instance is used as a cache, also running on Frodoi but utilizing its local SSD storage instead of the NFS-based storage pool. This setup enables faster access to frequently used data without relying on network communication. However,

since SSD storage is limited, we apply automatic retention policies to delete old cache records after a defined period so that we ensure efficient space management.

Monitoring

Monitoring components, including Loki, Fluentd, Grafana, and Prometheus, are deployed using Helm charts. They are configured to communicate with each other in the Kubernetes' internal network. Additionally, Prometheus is configured to scrape metrics from all system components, and Grafana is set up with pre-configured dashboards that show the health of the system, the resource usage of the services, and their performance metrics.

Emulators

Emulators are the supportive system components that are used to create workload emulations and evaluate the performance of the system. They are explained in detail in the Evaluation chapter (see Chapter 7). These emulators run as Kubernetes Statefulsets, ensuring that each instance acts as a distinct user with a unique and persistent identifier. Unlike standard Kubernetes deployments, Statefulsets guarantee that each instance retains its identity across restarts, allowing for controlled and repeatable emulations. In our case, multiple instances of them run simultaneously to emulate real-world usage patterns and assess system performance under different load conditions.

6.2.3 Service Configurations and Resource Limitations

To ensure secure and maintainable configuration management, we use Kubernetes ConfigMaps for non-sensitive configuration details and Secrets for storing sensitive data, such as private keys and authentication credentials. These configurations are passed to the services as environment variables, allowing for dynamic adjustments without modifying container images. For example, we have dynamic configurations, such as the batch size of saving data to the data lake, query timeout limits, and cache retention periods.

For resource limitations, we carefully allocate CPU and memory to prevent overloading any node. Each service is assigned resource constraints, ensuring that no single component consumes excessive resources. We aim to limit the total allocated resources to not exceed %80 of the node's CPU and memory capacity. This prevents crashes during peak loads and ensures stable system performance.

Since the backend and the data lake are the most resource-intensive components, they are allocated higher CPU and memory limits to ensure smooth operation of the system even under unexpectedly high loads. Kubernetes enforces these limitations, preventing any service from exceeding its allocated resources.

6.2.4 External Access

To enable secure and structured external access to our system, we utilize custom domain configurations, Kubernetes ingress resources, Nginx [57] as a reverse proxy, and encrypted HTTPs communication. These components work together to ensure a scalable, secure, and easily accessible pipeline.

Domains

Our primary domain is 'atasoyfurkan.live' and we use its subdomains to organize access to specific services. For example, 'grafana.pipeline.atasoyfurkan.live' subdomain is used for accessing the Grafana dashboard and 'api-backend.pipeline.atasoyfurkan.live' subdomain is used for accessing the APIs of the backend.

Ingresses

We use Kubernetes Ingress resources to manage external access to our deployed services. Each ingress rule is mapped to a specific service deployment, ensuring that requests to a given subdomain are correctly routed to the appropriate service. Our ingress setup minimizes the attack surface by only exposing the necessary services to the public network, which aligns with our security non-functional requirement (see Section 4.3.2).

Nginx

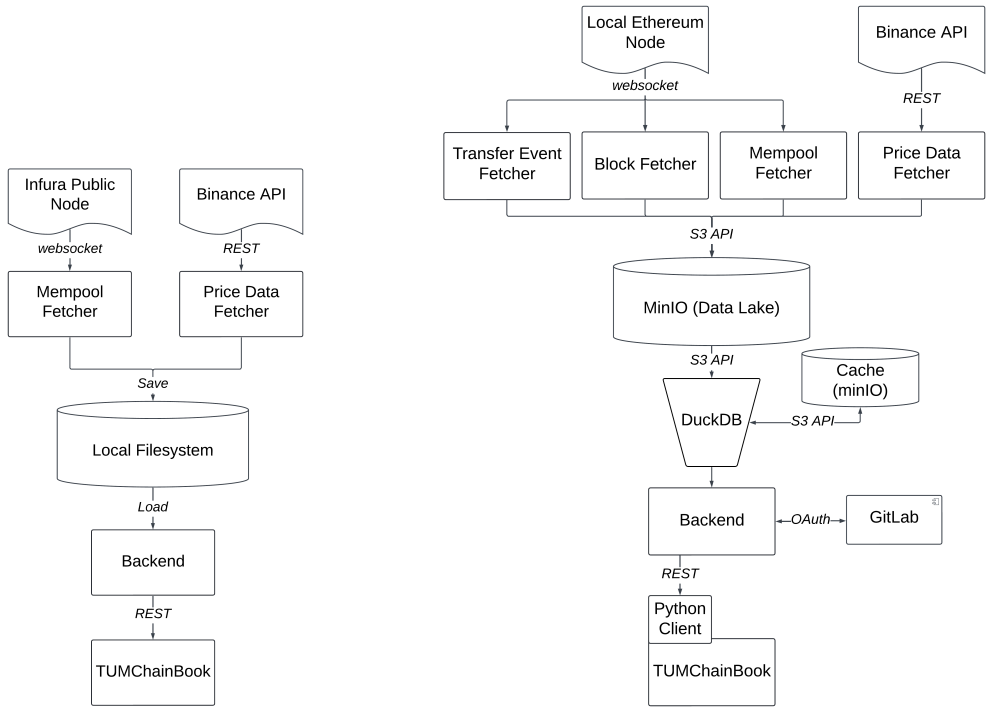
We use Nginx as a reverse proxy to manage incoming traffic. It is deployed via Helm charts and acts as a gateway for all external requests. It intercepts all incoming requests and determines their destination based on the subdomains they use. For example, a request to 'grafana.pipeline.atasoyfurkan.live' is forwarded to the Grafana ingress by looking at its 'grafana' subdomain definition.

Secure Communication

To ensure secure HTTPs connections, we integrate a certification manager resource in Kubernetes and connect it with Let's Encrypt [58] for automatic SSL/TLS certificate management. In this way, we automatically get new certificates and renew them when they expire, ensuring a secure connection to our services.

6.3 Iterative Development

Our development approach followed an iterative methodology, where we continuously refined and improved our pipeline over multiple iterations. Instead of attempting to design a perfect system from the beginning, we focused on building a simple but func-



(a) Architecture: First Version

(b) Architecture: Last Version

Figure 6.3: Iterative Development of the System

tional end-to-end version first, and then we gradually added improvements based on real-world observations and testing.

Iterative development methodology helped us in many ways. Instead of overplanning and misjudging actual needs, we continuously tested our assumptions and adapted the pipeline. In our iterations, we performed a workload analysis to identify bottlenecks and optimize performance dynamically. It also helped us to detect and resolve critical bugs early, preventing them from becoming deeply embedded in the system.

In total, we had four iterations, and in each iteration, we considerably improved our pipeline to make it production-ready and ready to process large-scale blockchain data.

6.3.1 Version 1

In the first iteration, our goal was to build a simple yet functional end-to-end system that could collect, store, and provide access to blockchain data for analysis. This version prioritized ease of development, using external third-party services instead of running our own infrastructure. Figure 6.3a illustrates the first version of our pipeline.

We integrated two primary data sources: Infura Public Node [59] to fetch blockchain data and Binance API to collect real-time price data. We used Infura's third-party service instead of running our self-hosted Ethereum node. We then implemented two fetchers, each responsible for gathering data from a respective data source. Then, the collected data was stored locally on disk in Parquet format. We used different folders for each data type we collected with our fetchers. Finally, our users could manually retrieve and load stored data into their TUMChainBook notebooks and execute basic queries on mempool and price data by running computations on the TUMChainBook side by manually joining datasets.

In this way, we created a working pipeline quickly with minimal infrastructure overhead.

6.3.2 Version 2

In the second iteration, we focused on improving the storage and access layers of our pipeline. The first version exposed several limitations, particularly in the usability and flexibility of data storage and retrieval. To address these, we introduced MinIO for storage, DuckDB as a query engine, and a Python client for better user interaction.

We replaced the local filesystem with MinIO. This change provided better reliability, maintainability, and extensibility since MinIO is an established solution with many out-of-the-box solutions (see Section 4.4.2). It implements data replication for fault tolerance and has built-in support for data processing tools.

We then introduced DuckDB to allow users to directly query data stored in MinIO using a familiar SQL-like interface. This eliminated the need for users to manually retrieve data files and process them within TUMChainBook. Users could now filter, aggregate, and join datasets efficiently within the pipeline itself instead of doing them in TUMChainBook.

We finally created a Python client to simplify user interaction with the backend API. The client encapsulates API calls and provides a user-friendly interface by abstracting away API complexities for end users.

6.3.3 Version 3

In the third iteration, we focused on improving data ingestion and ensuring production-ready data access. While the second iteration optimized storage and querying, the data collection process still had limitations, and there was no authentication mechanism. To address these, we expanded data sources, introduced a local blockchain node for reliability, and implemented secure user Authentication via OAuth and GitLab.

In the previous iterations, we relied on Infura's public Ethereum node, but their API has rate limitations, disallowing us to continuously ingest the blockchain data. To overcome this, we deployed our own local Ethereum node by using the Erigon implementation. It allowed unrestricted access to blockchain data. Additionally, we implemented additional fetchers to collect more diverse data, including transfer event and block data and also more detailed data for mempool and price data. With better data ingestion, we could now perform more accurate and meaningful analysis for our MEV use case. The pipeline now supports real-world transaction monitoring, price impact analysis, arbitrage detection, and many more, making it applicable for blockchain researchers.

To enable secure and controlled access to the pipeline, we integrated OAuth-based authentication via GitLab. The authentication process starts with users authenticating with GitLab in TUMChainBook, and then our Python client passes the resulting access token to our pipeline in every request, allowing the backend to validate and retrieve the user information by using this access token. Once authenticated, the backend retrieves the user's role (admin or researcher) and enforces specific permissions to users, such as query timeouts or cache retention timeouts.

6.3.4 Version 4

In the final iteration, we focused on testing, identifying bottlenecks, and optimizing the pipeline to ensure it is fully production-ready and scalable for multiple users. After implementing core functionalities in previous iterations, we conducted workload emulations, refined resource management, and introduced optimizations to improve system performance, stability, and efficiency.

We emulated real-word user workloads to stress-test the pipeline and detect weak points (see Chapter 7 for more information about workload emulations). According to the outputs of the emulations, we targeted the optimizations of the pipeline.

Firstly, we defined timeouts for DuckDB queries to prevent excessive resource usage from complex queries. Queries exceeding the allowed execution time are safely terminated, preventing system overload from inefficient or recursive queries. The implementation of the timeout is done manually since there is no built-in timeout support in DuckDB. We defined each query to run as a subprocess, allowing the system to track and terminate it if it exceeds the timeout limit.

Then, we defined Kubernetes resource limitations to ensure system stability. We configured deployments to in total only consume up to %80 of the CPU and memory of its node, preventing the system from crashing under high loads, which we encountered initially during workload emulation (see Section 6.2.3).

Finally, to reduce redundant computations and improve query performance, we implemented a caching mechanism. We created a new deployment of MinIO, which is running on the fast SSDs for our caching storage. Then, we implemented a new feature in our backend, allowing users to store and fetch the results of expensive queries. In this way, we avoid repeated recalculations and reduce load on the system, ensuring a more performance-efficient and scalable system.

Another bottleneck was the storage capacity, which did not make it possible to ingest all complex blockchain data continuously. Therefore, we integrated our MinIO data lake with NFS, allowing it to connect to our 40TB storage pool (see Section 6.2.1). This provided sufficient space to store high-volume blockchain data.

With these improvements, the pipeline became stable, scalable, and ready for real-world deployments. Figure 6.3b illustrates the final version of the pipeline.

Chapter 7

Evaluation

Evaluating the performance of our data pipeline is essential to ensure its efficiency, scalability, and real-world applicability. Our evaluation process is structured around three key areas:

1. *Performance Metrics:* We measure essential factors such as ingestion speed, query latency, and resource utilization. These metrics provide a clear understanding of how efficiently our pipeline processes and serves blockchain data
2. *Workload Emulation:* To test the pipeline under realistic conditions, we design workload emulations that mimic actual user behavior. Different user personas perform various types of blockchain analytics, allowing us to observe system performance under diverse query patterns.
3. *Optimizations:* Based on our observations, we identified bottlenecks that slow down the ingestion of our query execution and then, we implemented targeted optimizations to enhance the system's overall performance and maintainability.

7.1 Evaluation Metrics

To assess the efficiency and scalability of the pipeline, we define key evaluation metrics across three main areas: ingestion performance, query performance, and resource utilization. These metrics are selected based on both our workload characteristics and established practices in data-intensive system evaluation. For instance, blockchain research pipelines like ours must ingest continuously growing, high-throughput data streams, which makes ingestion latency and throughput essential metrics. Similarly, since researchers interact with the system via analytical queries, query latency and throughput

directly impact usability and responsiveness. Finally, resource usage metrics such as CPU and memory consumption are necessary to evaluate system efficiency under multi-user load. These metrics provide a structured approach to measure how well our system handles blockchain data processing and user queries.

7.1.1 Ingestion Metrics

This category evaluates how efficiently the pipeline collects, processes, and stores blockchain data.

- *Data Growth:* Total storage usage of our pipeline, including data lake storage and cache.
- *Ingestion Latency:* Time taken from data ingestion to availability for querying.
- *Ingestion Throughput:* Volume of data ingested per unit time

7.1.2 Query Metrics

This category focuses on how quickly and efficiently data is retrieved from the pipeline.

- *Query Latency:* Time taken for a query to execute and return results
- *Query Throughput:* Volume of data served by the backend per unit time.

7.1.3 Resource Usage Metrics

This category examines the computational resources used across different pipeline components.

- *CPU Usage:* Total CPU time used by the pipeline
- *Memory Usage:* Total memory consumption of the pipeline

7.2 Workload Emulation Setup

We designed our workload emulation with the aim of creating a realistic testing environment that closely mimics how researchers interact with the system. To achieve this, we structured the emulation with different user personas, query patterns, and caching behavior while ensuring diversity in workload distribution.

7.2.1 User Traffic

In this section, we focus on modeling user traffic to create a realistic workload emulation of our system.

Exponential Inter-Arrival Times

In our workload emulation, multiple users conduct their analyses by running Jupyter notebooks. We defined an inter-arrival time between two consecutive notebook runs by the same user, and we aimed to make these inter-arrival times as realistic as possible to model how the researcher would interact with our system (see Section 2.4.2).

Real-world research environments exhibit bursty and unpredictable user behavior. Some users frequently interact with the system, while others execute queries less often. We chose exponential inter-arrival times because they accurately model human behavior. Here are the key reasons for choosing exponential inter-arrival times:

1. Researchers do not submit queries at perfectly regular intervals. Instead, they may run multiple notebooks in a short time interval, followed by periods of inactivity. The exponential distribution naturally creates clusters of high activity followed by gaps, making it more realistic than uniform arrival times.
2. Exponential inter-arrival times measure system responsiveness under dynamic conditions since the system experiences periods of both high load (bursts) and low load (idle periods). In this way, we can better understand how the system behaves under different conditions.

Listing 7.1: Calculation of Sleep Time

```
1 import random
2 mean_interarrival_time = 3600 # in seconds
3 sleep_time = random.expovariate(1.0 / mean_interarrival_time)
```

We implement exponential inter-arrival times by waiting for a time drawn from an exponential distribution before each user executes their next notebook. This wait time is calculated using Listing 7.1. We configure the mean inter-arrival time to be 60 minutes to simulate a period when multiple researchers are in the critical phase of their research, frequently using the system to run analyses and retrieve data.

Open System Benchmarking

In performance testing, there are two primary benchmarking approaches: open system and closed system benchmarking (see Section 2.4.3 for a detailed explanation). We chose open system benchmarking since it better reflects unpredictable user behavior and allows us to test system scalability as new users arrive dynamically. We implemented open benchmarking by running each user's notebook independently of other users' progress, and we have no artificial limit on the number of concurrent users exist. For example, if many users run their notebooks at the same time, the system must handle it.

7.2.2 User Personas

In our workload emulation, we define different user personas to reflect realistic research behavior. Each researcher has a distinct goal, requiring them to run different notebooks and execute queries that mimic real-world blockchain analysis. By diversifying the emulation, we ensure that our benchmarking is accurate and reflective of how a blockchain research platform would be used. We have two user personas:

Trade Researchers: Their main research goals are to investigate market liquidity and transaction slippage and detect arbitrage opportunities and potential market manipulation. For that, they conduct the price impact analysis by getting pipeline data from transfer events and price data buckets.

Validation Researchers: Their main research goals are to analyze how transactions are validated, how gas fees influence prioritization, and to detect anomalies such as sandwich attacks. For that, they conduct gas price and priority trends analysis by getting pipeline data from block and mempool data buckets.

In total, we have 60 users running their queries with exponential inter-arrival times. There are 30 trade researchers and 30 validation researchers to ensure diversity in our emulation.

7.2.3 Workload Diversity

In our workload emulation, we ensure that users do not always query the same data range. Instead, for every notebook execution, a user is assigned a randomly chosen time range from our stored blockchain data, meaning each time a user runs their analysis, they are working with a different period, preventing repetitive access patterns.

By constantly changing the queried data range, we achieve a more diverse workload distribution across the system. This approach prevents overloading specific sections of our stored data while ensuring that our storage processing components handle a wide variety of queries.

7.2.4 Cache Behavior

In our emulation, we aimed to create a realistic cache behavior that reflects how users access and interact with stored data. To achieve this, we designed a dynamic cache hit rate influenced by three factors: continuous data ingestion, workload diversity, and cache usage probability.

Continuous Data Ingestion

Since our pipeline is constantly ingesting new data, a query cannot be served from the cache unless it has been previously requested and stored. This means that newly arriving data will initially result in cache misses, contributing to a realistic cache behavior.

Workload Diversity

As discussed in Section 7.2.3, we do not always fetch the same data ranges. Users query different time frames in each notebook execution, making it difficult for cache efficiency to be artificially high. This ensures that cache hits occur only when repeated requests exist, which is also valid for real-world user patterns.

Cache Usage Probability

To further model realistic behavior, we defined a cache usage probability, which acts like a static upper cache hit rate. Even if there is no continuous data ingestion and workload diversity, the overall cache hit rate cannot exceed this value. The implementation of the cache usage probability is done by not using the cache in our emulation, even if there is a valid record in the cache with the probability defined.

The most important part of the cache usage probability is deciding on a realistic value that reflects real-world usage so that we can create a realistic workload for our pipeline. Based on empirical research, cache hit rates typically range between 60% and 90% in various data-intensive applications. The Memento Aggregator Service [60], which archives web content, achieved 70-80% cache hit rates. The XRootD System [61], used for high-energy physics data retrieval, showed hit rates between 62% and 89%. Another study examining in-network storage caches for scientific workflows [62] showed that they have a 67.6% cache hit rate for their workload.

By looking at these empirical studies that have similar domains with our pipeline, we defined our static upper cache hit rate as 70%, and it is expected that our actual cache hit rate will be less since we also have other factors mentioned in the previous subsections that result in cache misses.

7.3 Workload Emulation Results

To evaluate the system's ability to handle continuous workloads, we ran our emulation for over a month without interruption. This long-term testing allowed us to observe how the pipeline performs under long-term researcher activity. By simulating real-world conditions, we ensured that our analysis captured both short-term performance fluctuations and long-term system stability. Please note that all of the data and visu-

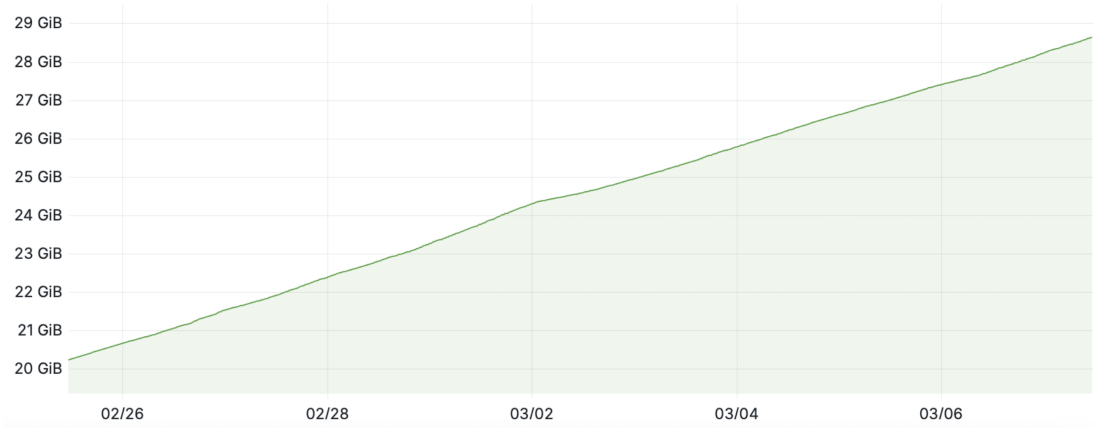


Figure 7.1: Data Growth Over Time

alizations are collected by using our monitoring components Grafana and Prometheus (see Section 5.2.4)

7.3.1 Data Growth

$$\begin{aligned} r_{\text{daily}} &= \frac{8.4 \text{ GB}}{10 \text{ days}} = 0.84 \text{ GB/day} \\ S_{\text{future}} &= 28.6 \text{ GB} + (0.84 \times 730) \text{ GB} = 641.8 \text{ GB} \end{aligned} \tag{7.1}$$

Monitoring data growth is essential to ensure our pipeline remains scalable and efficient. Since our data ingestion pipeline has been running continuously for over a month, we analyzed a recent 10-day period to estimate long-term storage needs using the most up-to-date data. The data growth plot is illustrated in Figure 7.1.

During this period, storage usage increased from 20.2 GB on February 25, 2025, to 28.6 GB on March 7, 2025, reflecting a growth of 8.4 GB over 10 days. Based on this trend, we computed the estimated daily growth rate and projected storage requirements for a two-year (730 days) period, as illustrated in Equation 7.1.

Table 7.1: Ingestion Latency and Throughput Metrics

Data Source	**Latency (s)**		**Throughput (KiB)**	
	Mean	**Max**	**Mean**	**Max**
Transfer Event Fetcher	51.4	100.2	0.768	1.20
Price Data Fetcher	55.3	85.2	0.30	0.335
Mempool Data Fetcher	54.3	79.0	3.15	5.06
Block Data Fetcher	54.5	56.2	5.80	12.5

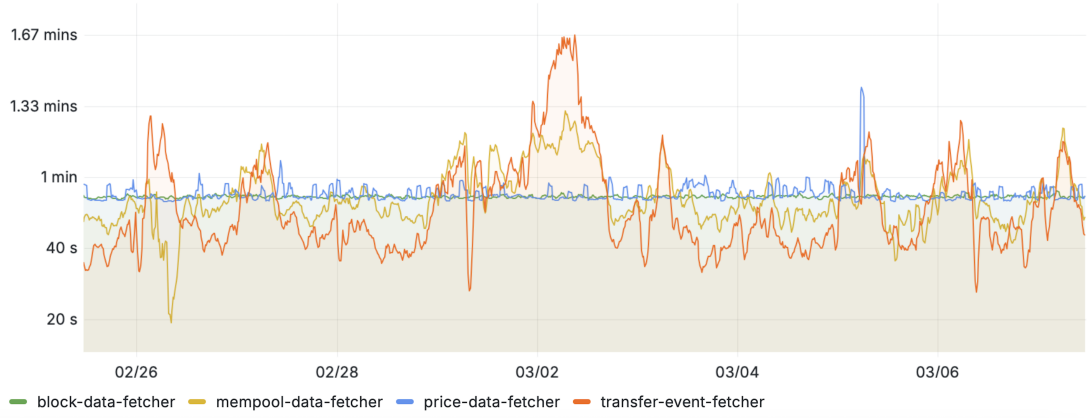


Figure 7.2: Ingestion Latencies by Data Source Over Time

This projection suggests that by maintaining the current data ingestion rate that is done for the MEV use case (see Section 4.1), our storage usage will reach approximately 642 GB in two years. To ensure reliability, we must also account for potential spikes in blockchain activity, metadata storage, and caching overhead. Given these uncertainties, we recommend provisioning at least 1 TB of storage. Moreover, these calculations are only done for the MEV use case, but in two two-year time spans, we might require more use cases and different analyses, such as P2P network analysis. Therefore, we need even more than 1 TB of storage.

7.3.2 Ingestion Latency and Throughput

To ensure real-time data processing, we configured ingestion latencies to be around 60 s for all fetchers, as shown in Table 7.1, where mean latency values range between 51.4 s and 55.3 s. This setup balances real-time data availability with system efficiency, ensuring users receive up-to-date blockchain and market data without long delays. To maintain this latency, we configured save batch sizes, which determine how many records are processed per cycle. Larger batch sizes improve throughput by reducing write overhead but can introduce delays, while smaller batches lower latency but increase system load due to frequent writes. By fine-tuning this trade-off, we achieved a stable ingestion rate that supports real-time access while maintaining system efficiency.

Figure 7.2 illustrates how ingestion latencies are changed during a 10-day period. There are fluctuations in mempool data and transfer event fetchers since they depend on Ethereum's real-time state, which means their ingestion latency fluctuates based on network congestion. On the other hand, the price data fetcher has a consistent ingestion latency since it is scheduled via cron jobs at fixed intervals. This regular scheduling

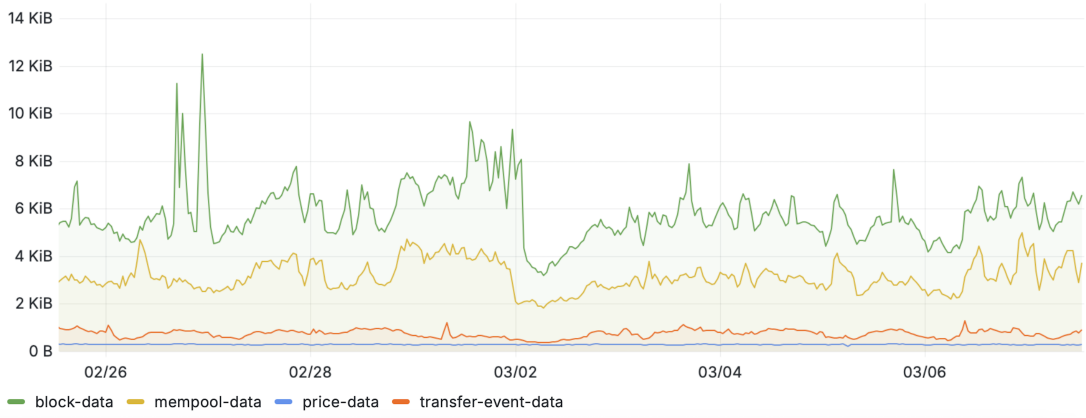


Figure 7.3: Ingestion Throughput by Data Source Over Time

ensures that latency does not fluctuate since it has fixed time intervals and fixed data save batch size.

Figure 7.3 illustrates how ingestion throughputs are changed during the same period. By examining also mean and max values in Table 7.1, we observe that block and mempool data fetchers have higher throughput compared to others because they have the same ingestion latency with others but have significantly larger data, and that makes the overall throughputs of them a lot higher.

7.3.3 CPU and Memory Usage

Table 7.2: CPU and Memory Usage Metrics

Service	**CPU Usage (cores)**		**Memory Usage (MiB)**	
	Mean	**Max**	**Mean**	**Max**
MinIO Datalake	3.51	3.78	18,000	18,700
API Backend	0.54	1.34	5,410	8,010
Transfer Event Fetcher	0.032	0.22	78.2	80.4
Price Data Fetcher	0.015	0.018	71.6	73.3
Mempool Fetcher	0.012	0.029	81.9	88.8
Block Fetcher	0.0016	0.0081	103.7	119

The metric results in Table 7.2 show that the system operates under continuous high load, with CPU and memory usage consistently near their limits. That means our current workload, running 30 trade researchers and 30 validation researchers (see Section 7.2.1), puts significant pressure on system resources, leading to higher query latencies and potential slowdowns.

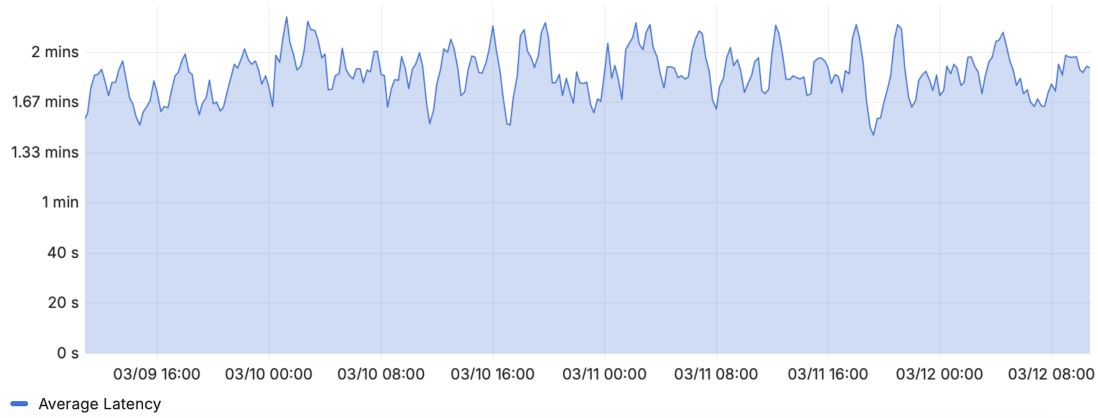


Figure 7.4: Query Latency Over Time

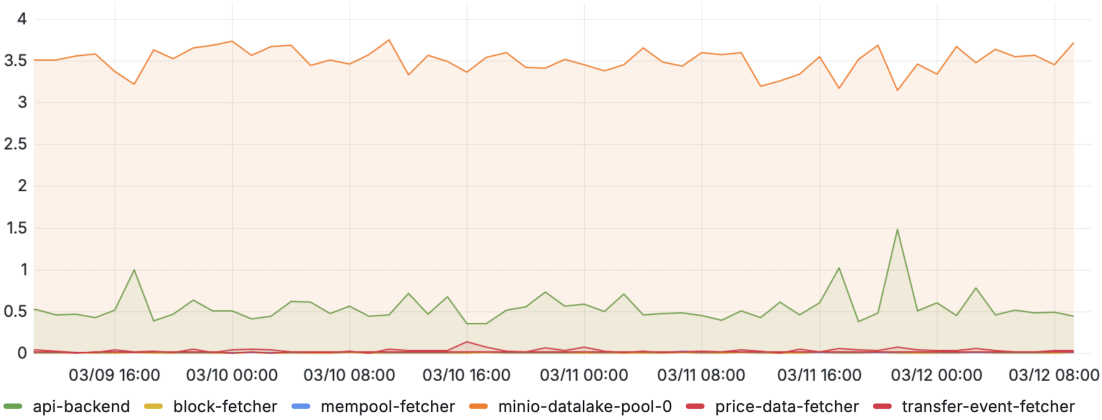


Figure 7.5: CPU Usages by Services Over Time

As Figure 7.5 illustrates, the CPU usage remains high, indicating that the system is constantly processing data without breaks. It causes our queries to run slower and have higher latencies, as shown in Figure 7.4. Therefore, it is clear that the current setup struggles to efficiently handle the workload, requiring optimizations to improve stability and performance.

7.3.4 Identified Bottlenecks

After running the workload emulation under the configured conditions, we identified the primary bottlenecks in our system. As observed in Table 7.2, MinIO data lake and API Backend, which runs DuckDB queries as sub-processes, consume the majority of the system's resources, making them the most critical components impacting overall performance. The high CPU and memory utilization of these services indicate that our

system is bottlenecked at data serving and query execution levels. Therefore, we need to improve their performance to make our pipeline performance efficient.

7.4 Performance Improvements

In this section, we discuss the performance improvements made to optimize the system. We focus on reducing bottlenecks, managing resource usage, and ensuring stable performance under high loads.

7.4.1 Reducing Load on Bottlenecks

As explained in Section 7.3.4), we have bottlenecks in our system. MinIO and API Backend use most of the resources of the node. They use a considerable amount of resources since when there is a complex query coming to the system, a lot of different objects are fetched from MinIO, and then they are joined, filtered, and processed in the API Backend using DuckDB, and these complex operations make the overall process resource-heavy.

We introduced a persistent caching layer to decrease the load of the system. It stores the results of the user queries and serves them back to the users when they want to run the same queries. In this way, it reduces the load of MinIO since the repetitive access to the same objects is not served by MinIO. The cache layer also reduces the load of the API Backend since it doesn't need to repeatedly perform complex joins and aggregations, leading to high CPU and memory usage.

Another critical factor contributing to MinIO's high load was its reliance on the Storage Pool that runs NFS to serve the files (see Section 6.2.1), which required data to be retrieved over the network. This added an extra layer of latency and resource usage. To overcome this, we deployed the cache on the fast SSD storage available on the Frodoi node (see Section 6.2.1). By doing so, we bypassed the need for frequent NFS-based storage access, ensuring faster read/write operations and improved data retrieval efficiency.

7.4.2 Preventing Resource Overload

Another critical issue in our system was node overload due to high request volume, leading to performance problems and, in extreme cases, system crashes. To prevent excessive resource consumption and ensure system stability, we implemented two key optimizations: resource limits for key services and query timeout to restrict overly complex queries.

Defined Resource Limits

After identifying MinIO and API backend as the primary workload components consuming most resources, we set resource limits to prevent the server from overloading. The goal was to allocate sufficient resources to critical services while ensuring system-wide stability. We ensured that total resource consumption never exceeded 80% of available capacity, leaving a buffer to handle unexpected load spikes and system operations. We also defined service-wise resource constraints and gave most of the resources to MinIO and API Backend, as they were the most demanding components.

Enforced Query Timeouts

Another major challenge was uncontrolled query execution, where users occasionally ran highly complex, resource-intensive queries that overwhelmed the system. Since there were no built-in query execution limits, a single inefficient or costly query could consume all of the resources, affecting all other users.

To avoid this, we implemented query timeouts in DuckDB, ensuring queries terminate if they exceed a defined execution time. This prevented resource starvation, ensuring that computational power remained fairly distributed, allowing multiple users to query the system simultaneously.

7.5 System Comparison

In this section, we compare different system configurations to evaluate their impact on performance. We analyze how NFS-backed storage affects ingestion and query latencies and assess the benefits of caching in reducing query times and system load.

7.5.1 Impact of NFS Storage

Table 7.3: Impact of NFS and Caching on Ingestion and Query Latencies

System Configuration	**Ingestion Latency (s)**		**Query Latency (s)**	
	Mean	**Max**	**Mean**	**Max**
Standard Storage (No NFS)	53.2	67.8	36.5	92.3
NFS-Backed Storage (No Cache)	53.3	70.2	112	134
NFS-Backed Storage with Caching	53.3	70.2	16.5	37.3

To evaluate the impact of NFS-backed storage, we compared MinIO running with NFS versus local SSD storage. We analyzed both ingestion and access parts of the pipeline. Table 7.3 shows the effects of the NFS-backed storage.

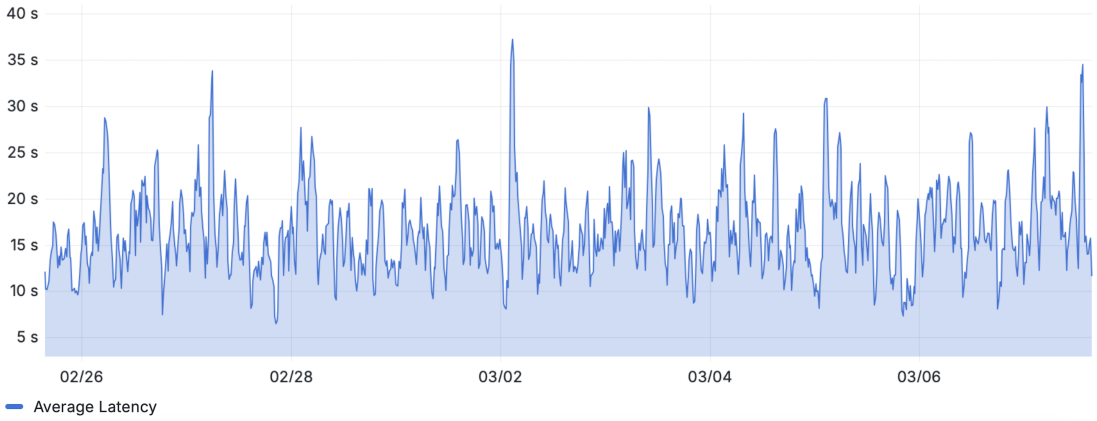


Figure 7.6: Query Latency Over Time (with Caching)

For ingestion latency, the results showed that NFS did not introduce a significant slowdown. The mean latency increased by only 0.1 s, which is an expected outcome since ingestion involves a single write operation to MinIO, and the overhead introduced by using network-attached storage is minimal.

In contrast, query performance was significantly affected. With NFS-backed MinIO, query latency increased five times compared to local SSD storage. The reason is that queries often require fetching many objects simultaneously, putting considerable stress on MinIO. Since NFS operates over a network, retrieving multiple records at once introduces additional latency and creates a high load on the system. In addition, as multiple queries run in parallel in our workload, additional network overhead increases delays, which makes query response times significantly longer.

Despite these performance drawbacks, the use of NFS-backed MinIO provides a significant advantage by increasing our overall storage capacity.

7.5.2 Impact of Caching

Table 7.4: Impact of Caching on Resource Consumption

Service	**Without Cache (CPU Cores)**		**With Cache (CPU Cores)**	
	Mean	**Max**	**Mean**	**Max**
MinIO Datalake	3.51	3.78	1.47	3.89
API Backend	0.54	1.34	0.29	1.83
MinIO Cache	-	-	0.15	1.12
Total	**4.05**	-	**1.91**	-

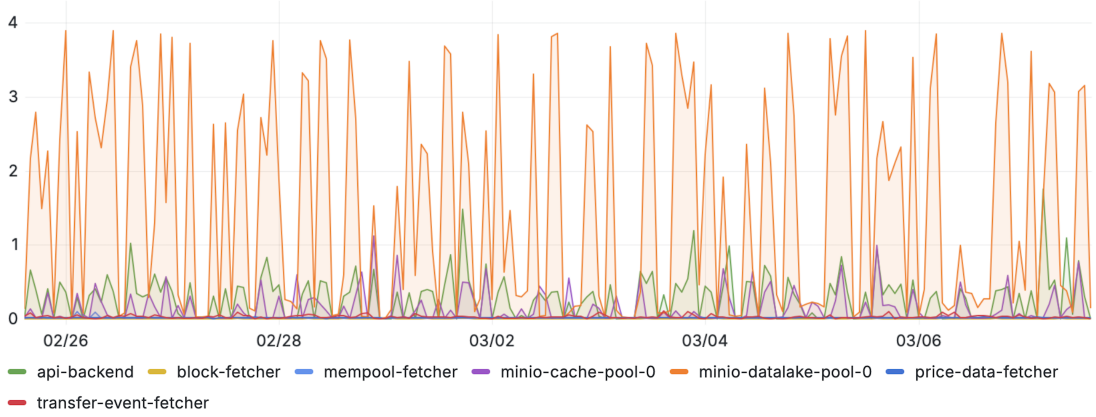


Figure 7.7: CPU Usages by Services Over Time (with Caching)

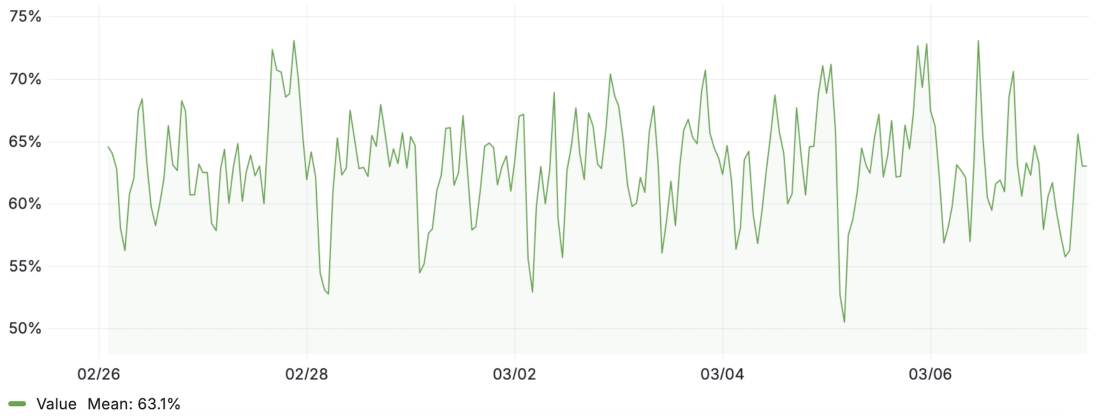


Figure 7.8: Cache Hit Rate Over Time

To evaluate the effect of caching, we compared the system's performance with and without a cache, both using MinIO-backed storage. The results clearly show that caching significantly improves query performance while also reducing overall system load. Table 7.3 shows the effects of caching on ingestion latency, and Table 7.4 shows the effects on the CPU utilization.

Caching has no effect on ingestion latency because it is only used for querying data. The ingestion process remains the same whether caching is enabled or not.

On the other hand, when we look at query performance, the difference is obvious. With caching, query latency is nearly 7 times faster. This improvement is even greater than the effects of using NFS-backed MinIO, effectively eliminating the downsides of it. Since caching reduces the number of direct requests to MinIO, the system can now handle queries efficiently, even when fetching data over the network.

Looking at the system's overall load, we can see the clear impact of caching. Figure 7.5 illustrates the CPU load of the system without caching, and Figure 7.7 illustrates the load with caching. Moreover, Figure 7.4 illustrates the query latency without caching, and Figure 7.6 illustrates the latency with cache. As shown in the figures, without caching, MinIO operates at maximum load continuously, struggling to handle queries. In contrast, with caching, peak loads only occur occasionally, during simultaneous cache misses. This means that most queries are efficiently served from the cache, keeping the overall system load much lower. This argument is further supported by the Table 7.4. As shown in it, caching reduces CPU load on both MinIO and the API backend by more than half. Even after adding a new MinIO cache component, the total CPU usage across all services is still less than half of what it was without caching. This confirms that caching significantly reduces system load, making the pipeline fully capable of handling the workload required for our use case.

Finally, to validate our cache hit rate assumption, which is discussed in Section 7.2.4, we monitored cache usage over time. We initially discussed that we capped the cache hit rate at 70%, and we were expecting it to be even lower because of continuous data ingestion and workload diversity. As observed in Figure 7.8, the actual cache hit rate fluctuates over time, with a mean value of 63%. This confirms that our workload emulation reflects real-world conditions.

7.6 Discussions

The evaluation results demonstrate that the pipeline can handle continuous blockchain data ingestion and serve multiple users concurrently under realistic workloads. Over the course of one month, the system successfully ingested new data in real-time, responded to user queries with acceptable latency, and remained stable under a load of 60 concurrent emulated users, 30 trade researchers and 30 validation researchers (see Section 7.2.1). This validates the system's design decisions and confirms that it meets the defined requirements.

7.6.1 Scalability and Requirements Fulfillment

Our findings show that the system fulfills both the functional and non-functional requirements defined in Section 4.3. The ingestion layer maintains real-time performance with ingestion latencies around 51.4 s (see Table 7.1), and the query layer remains responsive with mean query latencies as low as 16.5 s with caching enabled (see Table 7.3). Caching reduced total CPU consumption by more than 50% (see Table 7.4), showing that optimizations can unlock significant scalability gains.

Furthermore, the pipeline exhibits good maintainability and extensibility due to its modular ELT architecture. As discussed in Section 7.4, resource limits and query timeouts enhance robustness and support multi-user usage without system crashes. The use of schema-on-read and stateless processing tools like DuckDB simplifies adaptation to future data sources and schema changes.

7.6.2 Long-Term Operation and Planning

Given a projected storage growth rate of 0.84 GB/d, the system will require approximately 642 GB of storage in two years just for the MEV use case (see Equation 7.1). To ensure long-term viability, we recommend provisioning at least 1 TB of storage, with an even higher capacity if additional use cases are introduced, such as P2P network analysis.

The pipeline has proven its ability to operate continuously over a month without manual intervention, suggesting it is well-suited for production environments. Our evaluation showed that the pipeline used 1.91 CPU cores for 60 users and we currently have 8 CPU cores defined in our node. We want to utilize only 80% of our resources to ensure reliability and avoid system crashes. Therefore, we can at max use 6.4 CPU cores for our pipeline, which means we can serve three times more users. As a result, our pipeline is projected to serve 180 users simultaneously. Beyond this threshold, the pipeline can be horizontally scaled to multiple nodes and be able to serve even more users.

Chapter 8

Conclusion

Blockchain data is inherently complex, high-volume, and constantly evolving, making it challenging to develop a system that can continuously ingest, process, and serve this data efficiently. Existing solutions, such as public RPC clients and platforms like Dune Analytics, provide partial solutions but lack flexibility or full access to both on-chain and off-chain data. This thesis addresses these limitations by developing a production-ready blockchain data pipeline capable of long-term, scalable, and efficient data ingestion while offering researchers a user-friendly interface through JupyterHub integration.

8.1 Research Questions

We aimed to answer key research questions regarding the design, implementation, and evaluation of a production-ready blockchain data pipeline. In the following section, we summarize how each question was addressed through our design choices.

RQ1: *How can we design and deploy a pipeline that can continuously ingest blockchain data over long periods?*

To ensure long-term data ingestion, the pipeline must meet several non-functional requirements:

Scalability: As blockchain data grows exponentially, the system must scale to handle increasing storage needs and continuous user access without bottlenecks. Our data lake-based approach ensures that the system can store and retrieve large datasets efficiently.

Simplicity & Maintainability: A maintainable system is crucial for long-term operation. By using established open-source solutions (MinIO, DuckDB, and Kubernetes), the pipeline remains developer-friendly and easy to operate.

Performance Efficiency: Continuous operation means increasing costs, so the system must optimize compute and storage resources. We implemented caching mechanisms to reduce redundant queries, improving query performance while lowering resource usage.

Extensibility: Blockchain structures evolve, and new chains emerge. The pipeline's modular design allows easy adaptation to new blockchain schemas and integration of additional data sources without requiring major architectural changes.

Observability: Effective monitoring ensures smooth operation. We integrated resource monitoring and logging into the pipeline, allowing for real-time visibility into ingestion performance and system health.

By meeting these requirements, we ensure the pipeline is sustainable, reliable, and capable of handling blockchain data ingestion for long periods.

RQ2: *Which established technologies and solutions should be used to make the pipeline robust and production-ready?*

To develop a production-ready pipeline, we use different established solutions:

MinIO as the data lake: It is chosen for its scalability, community support, and object storage capabilities, making it ideal for large-scale blockchain data storage.

DuckDB as the query engine: It offers SQL-like functionality, high efficiency for analytical workloads, and seamless integration with MinIO, making it a lightweight yet powerful solution.

JupyterHub for researcher accessibility: It provides a multi-user, scalable platform for researchers to analyze blockchain data using notebooks, ensuring an intuitive and customizable data exploration experience.

Kubernetes for orchestration: It is a widely recognized tool that enables scalability and automated deployment of pipeline components, ensuring high availability.

By leveraging these technologies, we ensure that the pipeline is robust, scalable, and easy to maintain while minimizing operational complexity.

RQ3: *What is the most suitable architecture for building a large-scale data collection pipeline?*

We adopted an Extract, Load, Transform (ELT) architecture with caching mechanisms, as it offers greater flexibility compared to traditional ETL, particularly for handling blockchain data, which is semi-structured and continuously evolving. By first storing raw data and transforming it later, our approach ensures adaptability in data processing and allows schema changes over time without down times. Additionally, using a data

lake structure instead of a traditional data warehouse allows for high scalability, supporting both structured and unstructured blockchain data. Finally, implementing caching mechanisms significantly reduces the workload on storage and compute resources, improving query performance and making data retrieval faster and more resource-efficient.

RQ4: *Which representative use cases are suitable for demonstrating the effectiveness of the pipeline?*

MEV is a critical blockchain research topic, requiring high-frequency transaction data and real-time insights, making it an ideal test case for our pipeline. By executing an MEV analysis within our JupyterHub-integrated environment, we demonstrated that the pipeline could ingest, process, and analyze complex blockchain data efficiently.

8.2 Future Work

While the implemented pipeline meets the research objectives, several enhancements can further improve its efficiency, scalability, and usability. This section explains key areas where future developments can be done.

8.2.1 Expanding Data Sources

Currently, the pipeline includes MEV analysis use case, to test its effectiveness. Expanding these use cases would help us better understand blockchain workloads and how users interact with the system. One key area to explore is P2P network analysis, which would provide insights into network behavior, transaction propagation, and peer communication patterns.

Adding more diverse workloads would also allow us to test the system under different conditions. For example, introducing resource-intensive tasks like P2P network monitoring would help evaluate how well the pipeline handles high data loads and iteratively optimize it. This would improve the system's efficiency and ensure it remains stable and effective for a wider range of blockchain research needs.

8.2.2 Metadata Management and Improved Data Exploration

To make the pipeline easier to use and manage, integrating established solutions for data organization and workflow automation would be helpful. Using a data cataloging system like Apache Atlas would improve metadata management, making it simpler for researchers to find, track, and organize datasets. It would also help in tracking data changes over time, making it clear how data is collected and processed in the pipeline.

Additionally, a more user-friendly interface for exploring data would allow researchers to access and analyze information more easily without relying on complex manual queries.

8.2.3 Dynamic Caching Strategies

The current caching system mainly uses persistent queries, where users manually save frequently accessed data to reduce query load. While this works well, an automated caching system could improve performance even more. A dynamic caching strategy that tracks query patterns and resource usage could manage the cache more efficiently. By using these insights, the system could automatically decide which queries and datasets to cache and adjust caching rules based on real-time demand.

8.3 List of acronyms

PoW Proof of Work
JSON JavaScript Object Notation
CSV Comma-Separated Values
ML Machine Learning
ELT Extract, Load, Transform
ETL Extract, Transform, Load
HDFS Hadoop Distributed File System
BI Business Intelligence
SQL Structured Query Language
NFS Network File System
REST Representational State Transfer
JWT JSON Web Token
RBAC Role-Based Access Control
MEV Maximal Extractable Value
DEXs Decentralized Exchanges
CEXs Centralized Exchanges

Bibliography

[1] Dune Analytics, *Dune analytics: Crypto's data platform*, Accessed: 2025-02-22, 2025. [Online]. Available: `https://dune.com/home`.

[2] D. Puthal, N. Malik, S. P. Mohanty, E. Kougianos, and G. Das, "Everything you wanted to know about the blockchain: Its promise, components, processes, and problems", *IEEE Consumer Electronics Magazine*, vol. 7, no. 4, pp. 6–14, 2018.

[3] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system", *Satoshi Nakamoto*, 2008.

[4] Z. Zheng, S. Xie, H. Dai, X. Chen, and H. Wang, "An overview of blockchain technology: Architecture, consensus, and future trends", in *2017 IEEE international congress on big data (BigData congress)*, Ieee, 2017, pp. 557–564.

[5] S. Cao, T. Miller, M. Foth, W. Powell, X. Boyen, and C. Turner-Morris, "Integrating on-chain and off-chain governance for supply chain transparency and integrity", *arXiv preprint arXiv:2111.08455*, 2021.

[6] V. Buterin *et al.*, "Ethereum white paper", *GitHub repository*, vol. 1, pp. 22–23, 2013.

[7] F. Vogelsteller and V. Buterin, *Eip-20: Erc-20 token standard*, Accessed: 2025-02-17, 2015. [Online]. Available: `https://eips.ethereum.org/EIPS/eip-20`.

[8] M. Shen, L. Zhu, and K. Xu, "Blockchain and data sharing", pp. 15–27, 2020. DOI: `10.1007/978-981-15-5939-6_2`.

[9] S. S. Chawathe, "Clustering blockchain data", *Clustering Methods for Big Data Analytics: Techniques, Toolboxes and Applications*, pp. 43–72, 2019.

[10] S. K. Behfar, F. Théodoloz, C. Schranz, and M. Hosseinpour, "Blockchain-based data sharing platform customization with on/off-chain data balancing", in *2023 Fifth International Conference on Blockchain Computing and Applications (BCCA)*, IEEE, 2023, pp. 487–494.

[11] C. Mathis, "Data lakes", *Datenbank-Spektrum*, vol. 17, no. 3, pp. 289–293, 2017.

[12] F. Ravat and Y. Zhao, "Data lakes: Trends and perspectives", in *Database and Expert Systems Applications: 30th International Conference, DEXA 2019, Linz, Austria, August 26–29, 2019, Proceedings, Part I 30*, Springer, 2019, pp. 304–313.

[13] M. Zaharia, R. S. Xin, P. Wendell, *et al.*, "Apache spark: A unified engine for big data processing", *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[14] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The hadoop distributed file system", in *2010 IEEE 26th symposium on mass storage systems and technologies (MSST)*, Ieee, 2010, pp. 1–10.

[15] Amazon Web Services, *Amazon simple storage service (s3) documentation*, 2025. [Online]. Available: `https://docs.aws.amazon.com/s3/index.html`.

[16] MinIO, Inc., *Minio documentation*, 2025. [Online]. Available: `https://min.io/docs.html`.

[17] B. Devlin, *Data warehouse: from architecture to implementation*. Addison-Wesley Longman Publishing Co., Inc., 1996.

[18] W. H. Inmon, "What is a data warehouse?", *Prism Tech Topic*, vol. 1, no. 1, pp. 1–5, 1995.

[19] Amazon Web Services, *Amazon redshift documentation*, 2025. [Online]. Available: `https://docs.aws.amazon.com/redshift/`.

[20] Snowflake Inc., *Snowflake documentation*, 2025. [Online]. Available: `https://docs.snowflake.com/`.

[21] B. Dageville, T. Cruanes, M. Zukowski, *et al.*, "The snowflake elastic data warehouse", in *Proceedings of the 2016 International Conference on Management of Data*, 2016, pp. 215–226.

[22] ClickHouse, Inc., *Clickhouse documentation*, 2025. [Online]. Available: `https://clickhouse.com/docs/en/`.

[23] S. A. Errami, H. Hajji, K. A. El Kadi, and H. Badir, "Spatial big data architecture: From data warehouses and data lakes to the lakehouse", *Journal of Parallel and Distributed Computing*, vol. 176, pp. 70–79, 2023.

[24] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, "Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics", in *Proceedings of CIDR*, vol. 8, 2021, p. 28.

[25] R. Sandberg, D. Goldberg, S. Kleiman, D. Walsh, and B. Lyon, "Design and implementation of the sun network filesystem", in *Proceedings of the Summer USENIX Conference*, USENIX Association, 1985, pp. 119–130. [Online]. Available: `https://web.stanford.edu/class/cs240/readings/nfs.pdf`.

[26] M. Raasveldt and H. Mühleisen, "Duckdb: An embeddable analytical database", in *Proceedings of the 2019 International Conference on Management of Data*, 2019, pp. 1981–1984.

[27] S. Patni, *Pro RESTful APIs*. Springer, 2017.

[28] I. Fette and A. Melnikov, "The websocket protocol", Tech. Rep., 2011.

[29] E. D. Hardt, *The oauth 2.0 authorization framework*, Internet Engineering Task Force (IETF) Request for Comments, 2012. DOI: 10.17487/RFC6749. [Online]. Available: https://www.rfc-editor.org/info/rfc6749.

[30] Auth0, *Role-based access control*, Accessed: 2025-02-20, 2025. [Online]. Available: https://auth0.com/docs/manage-users/access-control/rbac.

[31] The Prometheus Authors, *Prometheus documentation*, Accessed: 2025-02-20, 2025. [Online]. Available: https://prometheus.io/docs/introduction/overview/.

[32] Grafana Labs, *Grafana loki documentation*, Accessed: 2025-02-20, 2025. [Online]. Available: https://grafana.com/docs/loki/latest/.

[33] Fluentd Project, *Fluentd documentation*, Accessed: 2025-02-20, 2025. [Online]. Available: https://docs.fluentd.org/.

[34] S. S. Patil and S. S. Patil, "A comprehensive study of elasticsearch", *International Journal of Science and Research (IJSR)*, vol. 10, no. 6, pp. 1276–1280, 2021. [Online]. Available: https://www.ijsr.net/archive/v10i6/SR21529233126.pdf.

[35] Elastic, *Logstash reference*, Accessed: 2025-02-20, 2025. [Online]. Available: https://www.elastic.co/guide/en/logstash/current/index.html.

[36] M. A. M. Capretz and F. Ahmed, "A new approach based on elk stack for the analysis and visualization of geo-referenced sensor data", in *SN Computer Science*, 2022. [Online]. Available: https://dl.acm.org/doi/10.1007/s42979-022-01628-6.

[37] Grafana Labs, *Grafana documentation*, Accessed: 2025-02-20, 2025. [Online]. Available: https://grafana.com/docs/grafana/latest/.

[38] R. Han, L. K. John, and J. Zhan, "Benchmarking big data systems: A review", *IEEE Transactions on Services Computing*, vol. 11, no. 3, pp. 580–597, 2017.

[39] G. Casale, "Building accurate workload models using markovian arrival processes", *ACM SIGMETRICS Performance Evaluation Review*, vol. 39, no. 1, pp. 357–358, 2011.

[40] B. Schroeder, A. Wierman, and M. Harchol-Balter, "Open versus closed: A cautionary tale", 2006.

[41] I. Ayvaci, "Data collection pipeline for blockchain platforms", Master's Thesis, Technical University of Munich, 2024.

[42] P. J. Contributors, *Zero to jupyterhub with kubernetes*, Accessed: 2025-02-22, 2025. [Online]. Available: https://z2jh.jupyter.org/.

[43] B. Weintraub, C. F. Torres, C. Nita-Rotaru, and R. State, "A flash (bot) in the pan: Measuring maximal extractable value in private pools", in *Proceedings of the 22nd ACM Internet Measurement Conference*, 2022, pp. 458–471.

[44] T. Chi, N. He, X. Hu, and H. Wang, "Remeasuring the arbitrage and sandwich attacks of maximal extractable value in ethereum", *arXiv preprint arXiv:2405.17944*, 2024.

[45] B. Öz, F. Rezabek, J. Gebele, F. Hoops, and F. Matthes, "A study of mev extraction techniques on a first-come-first-served blockchain", in *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*, 2024, pp. 288–297.

[46] C. Ferreira Torres, A. Mamuti, B. Weintraub, C. Nita-Rotaru, and S. Shinde, "Rolling in the shadows: Analyzing the extraction of mev across layer-2 rollups", in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 2591–2605.

[47] Ethereum Foundation, *Ethereum the merge*, Accessed: 2025-02-22, 2022. [Online]. Available: `https://ethereum.org/en/roadmap/merge/`.

[48] E. Contributors, *Erigon: Ethereum implementation on the efficiency frontier*, Accessed: 2025-02-22, 2025. [Online]. Available: `https://github.com/erigontech/erigon`.

[49] Paradigm, *Reth: Modular, contributor-friendly, and blazing fast ethereum client*, Accessed: 2025-02-22, 2025. [Online]. Available: `https://github.com/paradigmxyz/reth`.

[50] G. E. Team, *Geth: Go ethereum official client*, Accessed: 2025-02-22, 2025. [Online]. Available: `https://geth.ethereum.org/`.

[51] QuickNode, *Ethereum full node vs. archive node*, Accessed: 2025-02-23, 2023. [Online]. Available: `https://www.quicknode.com/guides/infrastructure/node-setup/ethereum-full-node-vs-archive-node`.

[52] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing", in *Proceedings of the NetDB (Networked Systems Design and Implementation) Workshop*, Cambridge, MA, 2011. [Online]. Available: `https://cs.brown.edu/courses/cs227/archives/2014/papers/kafka.pdf`.

[53] The Kubernetes Authors, *Kubernetes documentation*, Accessed: 2025-02-27, 2024. [Online]. Available: `https://kubernetes.io/docs/`.

[54] T. R. L. Team, *K3s - lightweight kubernetes*, Accessed: 2025-02-27, 2024. [Online]. Available: `https://k3s.io/`.

[55] S. Merkel, *Docker: Lightweight Linux Containers for Consistent Development and Deployment*. Docker Inc., 2014, Accessed: 2025-02-27. [Online]. Available: `https://docs.docker.com/`.

[56] The Helm Authors, *Helm - the kubernetes package manager*, Accessed: 2025-02-27, 2024. [Online]. Available: https://helm.sh/docs/.

[57] W. Reese, "Nginx: The high-performance web server and reverse proxy", *Linux Journal*, vol. 2008, no. 173, p. 2, 2008.

[58] J. Aas, R. Barnes, B. Case, *et al.*, "Let's encrypt: An automated certificate authority to encrypt the entire web", in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 2473–2487.

[59] Infura, *Infura - scalable blockchain infrastructure*, Accessed: 2025-02-27, 2025. [Online]. Available: https://www.infura.io/.

[60] M. Klein, L. Balakireva, and H. Shankar, *Evaluating memento service optimizations*, 2019. arXiv: 1906.00058 [cs.IR]. [Online]. Available: https://arxiv.org/abs/1906.00058.

[61] J. Bellavita, A. Sim, K. Wu, *et al.*, "Studying scientific data lifecycle in on-demand distributed storage caches", in *Fifth International Workshop on Systems and Network Telemetry and Analytics*, ser. HPDC '22, ACM, Jun. 2022, 43–50. DOI: 10.1145/3526064.3534111. [Online]. Available: http://dx.doi.org/10.1145/3526064.3534111.

[62] C. Sim, K. Wu, A. Sim, *et al.*, "Effectiveness and predictability of in-network storage cache for scientific workflows", in *2023 International Conference on Computing, Networking and Communications (ICNC)*, IEEE, Feb. 2023, 226–230. DOI: 10.1109/icnc57223.2023.10074058. [Online]. Available: http://dx.doi.org/10.1109/ICNC57223.2023.10074058.